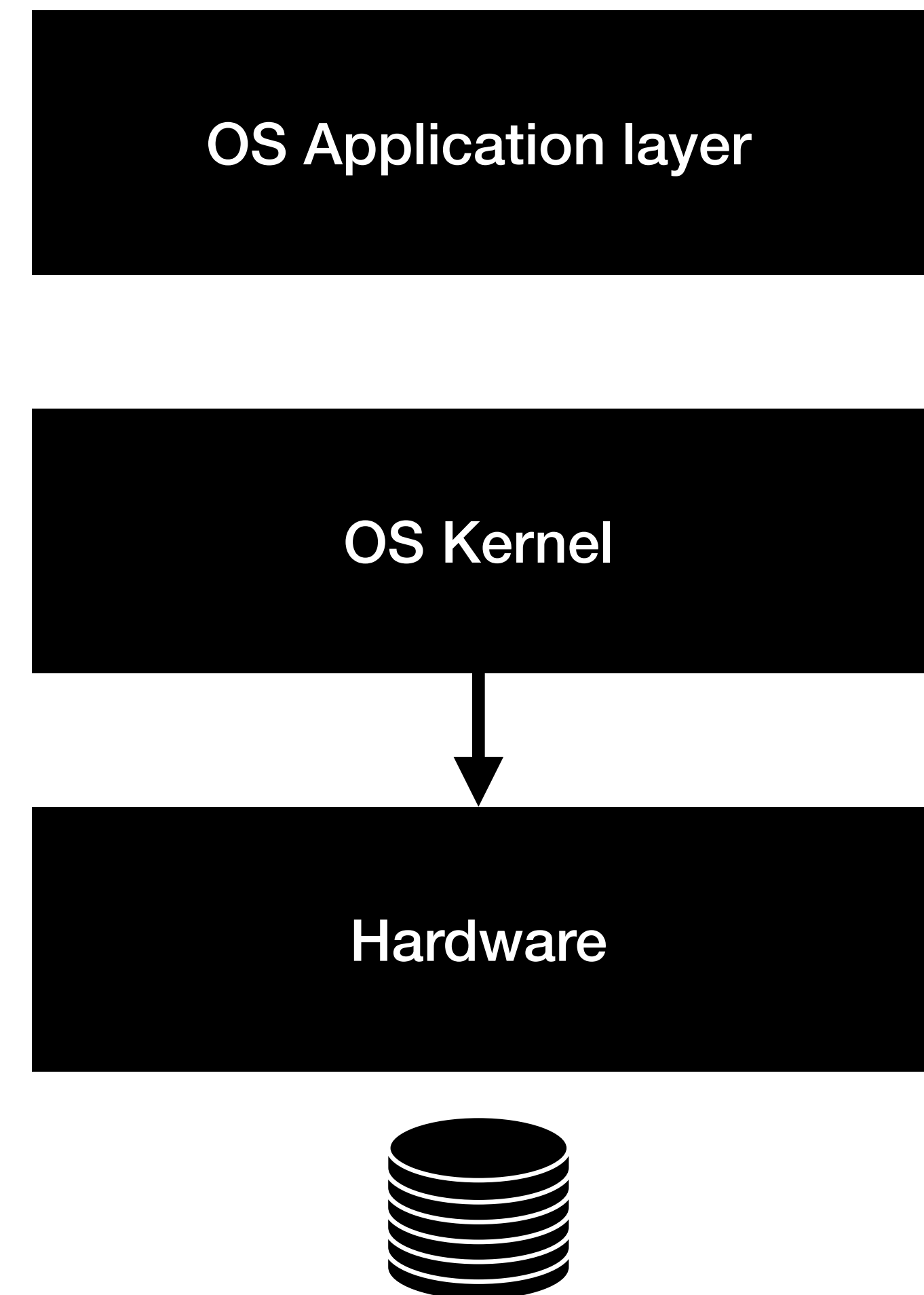


Vsebniki - zabojniki

Containers

Vsebniki vs. VM

- Kernel je jedro sistema
- Jedro (kernel) alocira vse zahtevane komponente



Vsebniki vs. VM

Kateri del se virtualizira?



OS Application layer

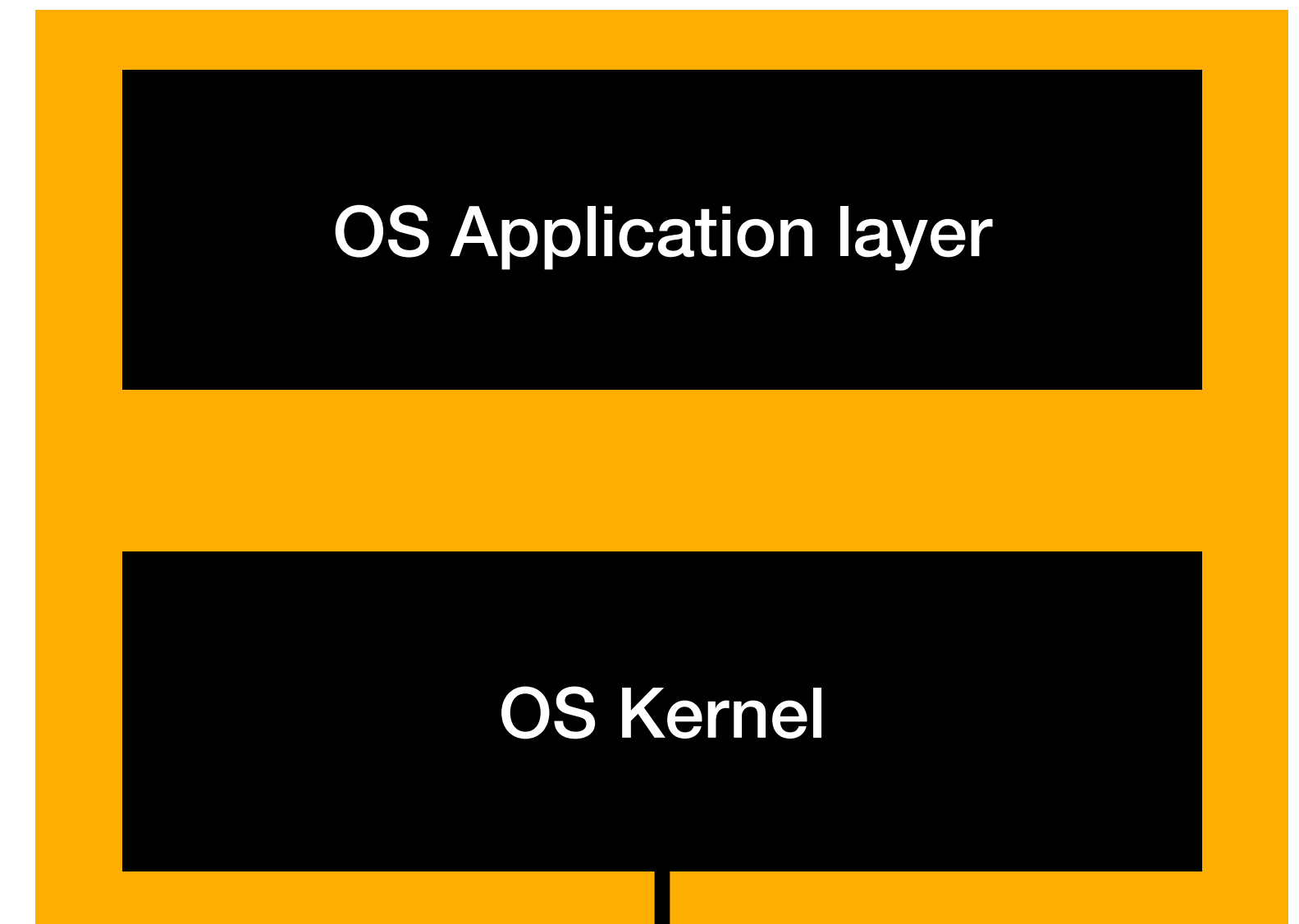
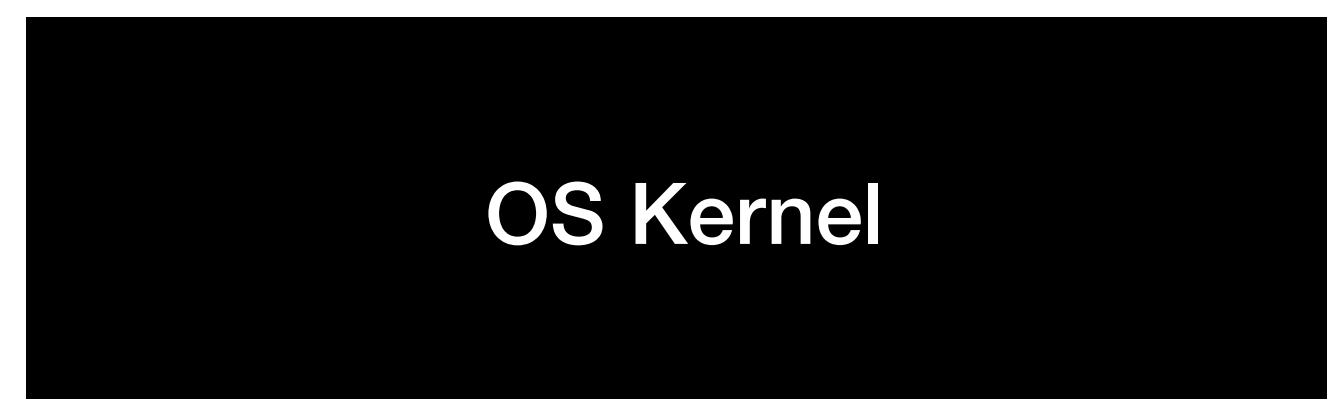
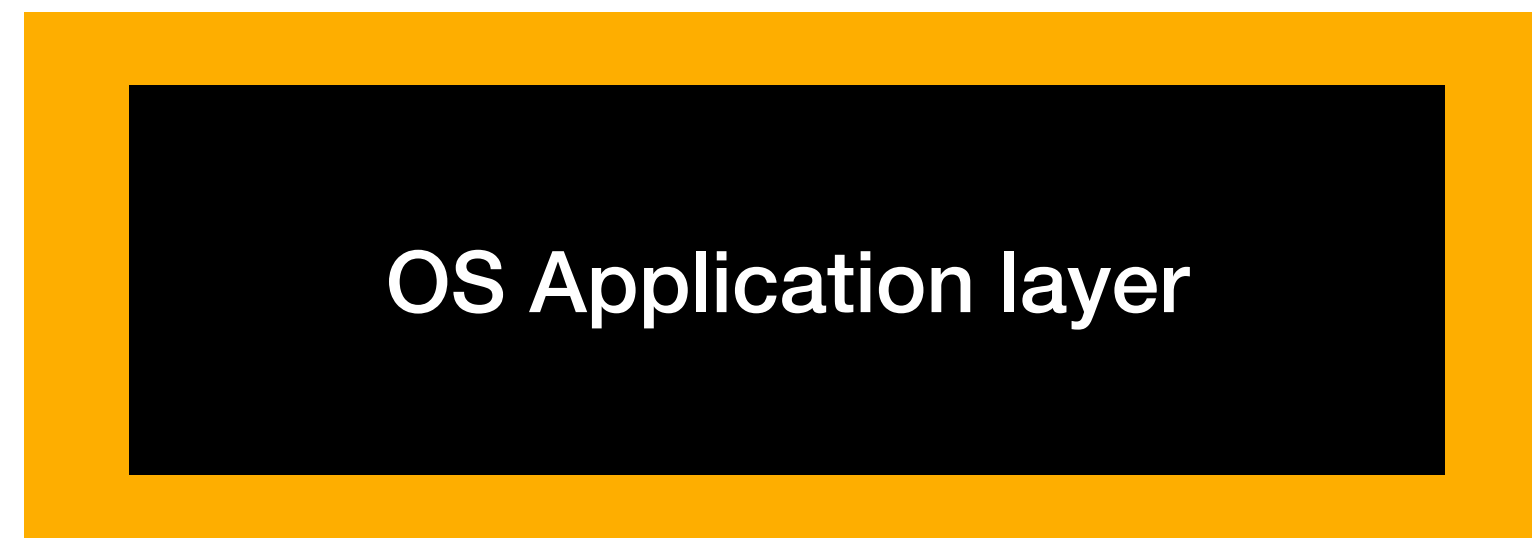
OS Kernel

Hardware

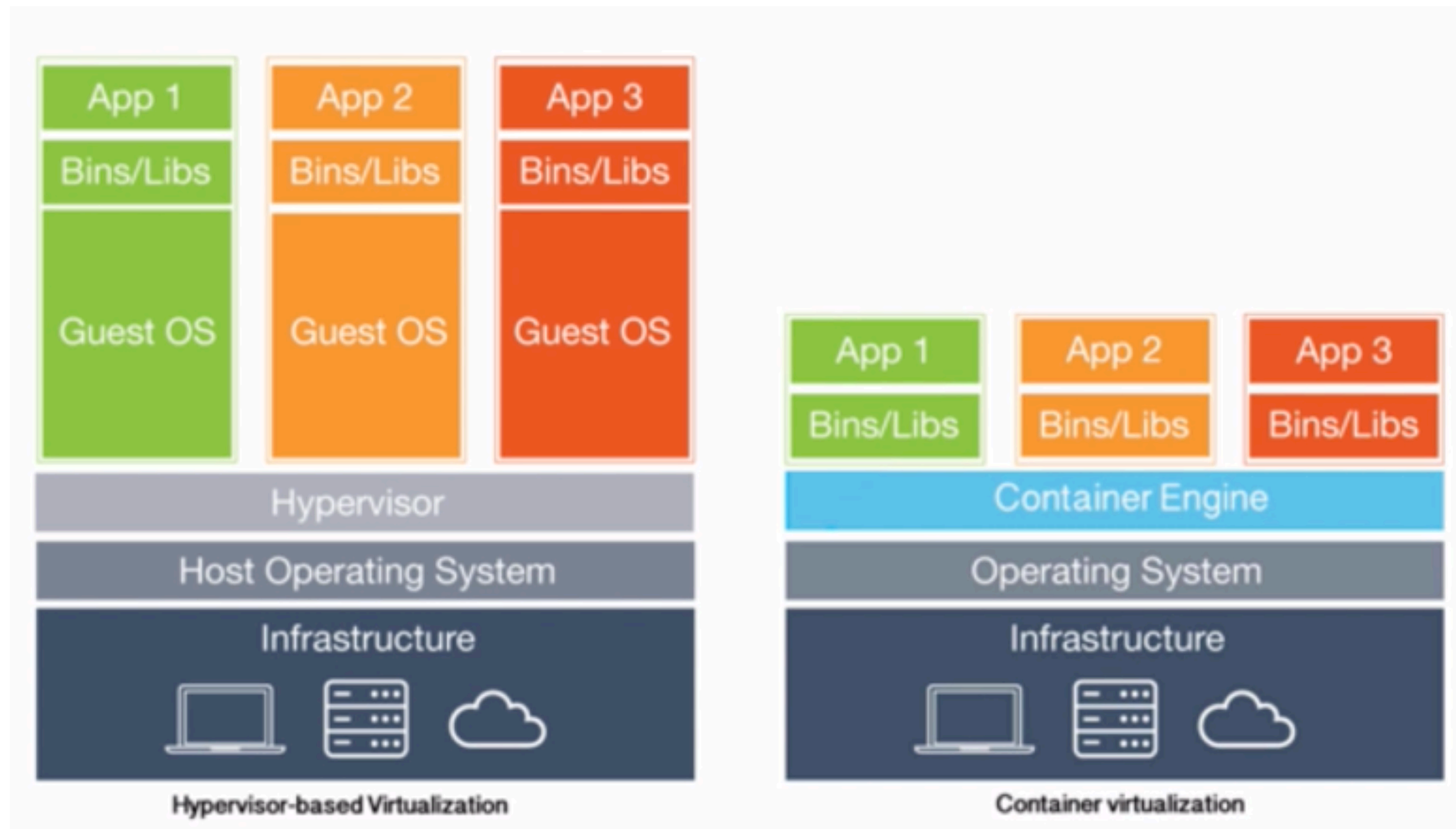
- Vsebuje OS Application layer
- Dodatne servise in aplikacije, ki so v podporo

Vsebniki vs. VM

Kateri del se virtualizira?



Vsebniki vs. VM



Vsebniki vs. VM

Kateri del se virtualizira?



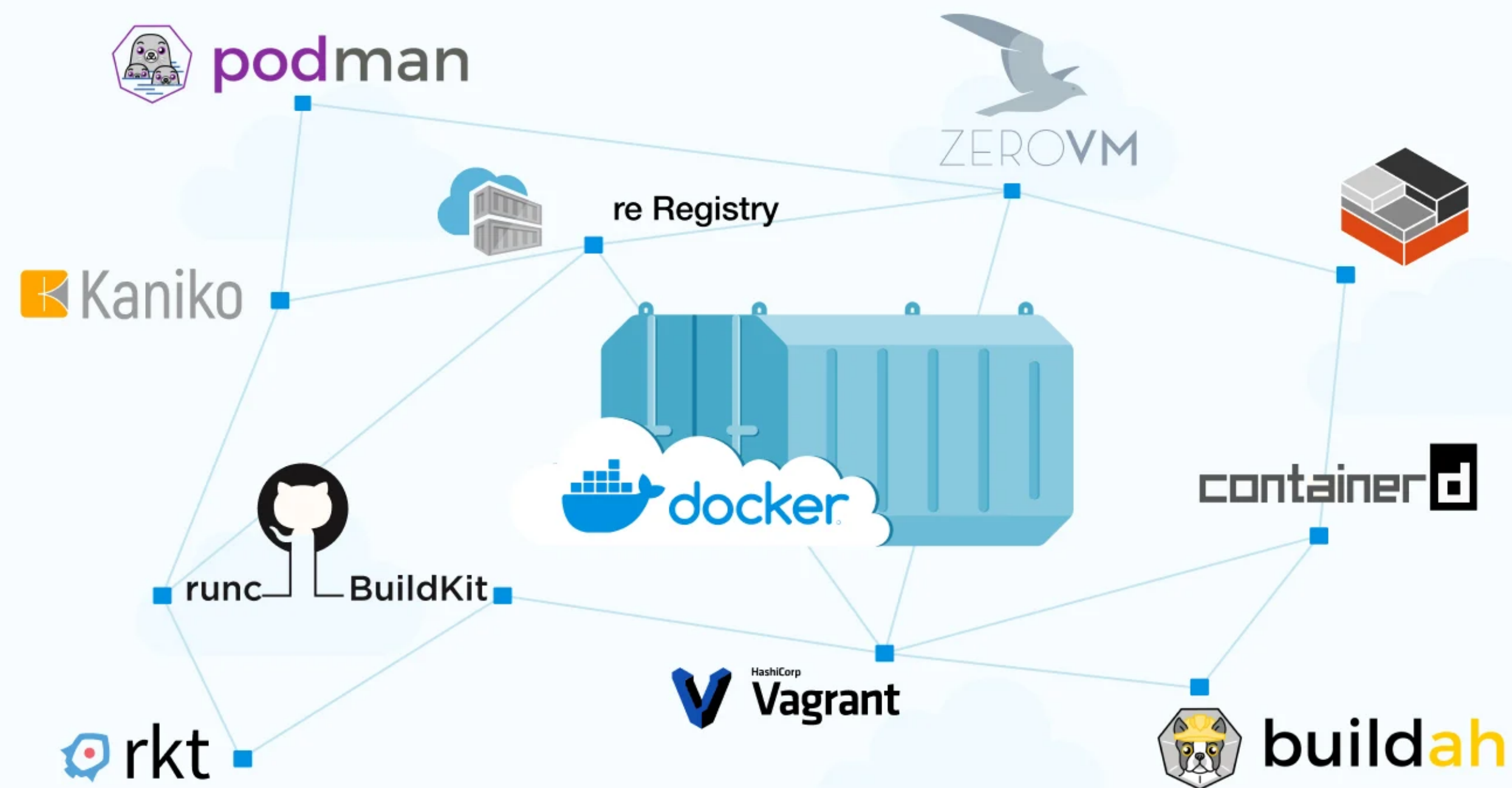
- Vsebniki so manjši
- Hitreje jih lahko štartamo in izklapljamo



- Lahko delajo na različne OS

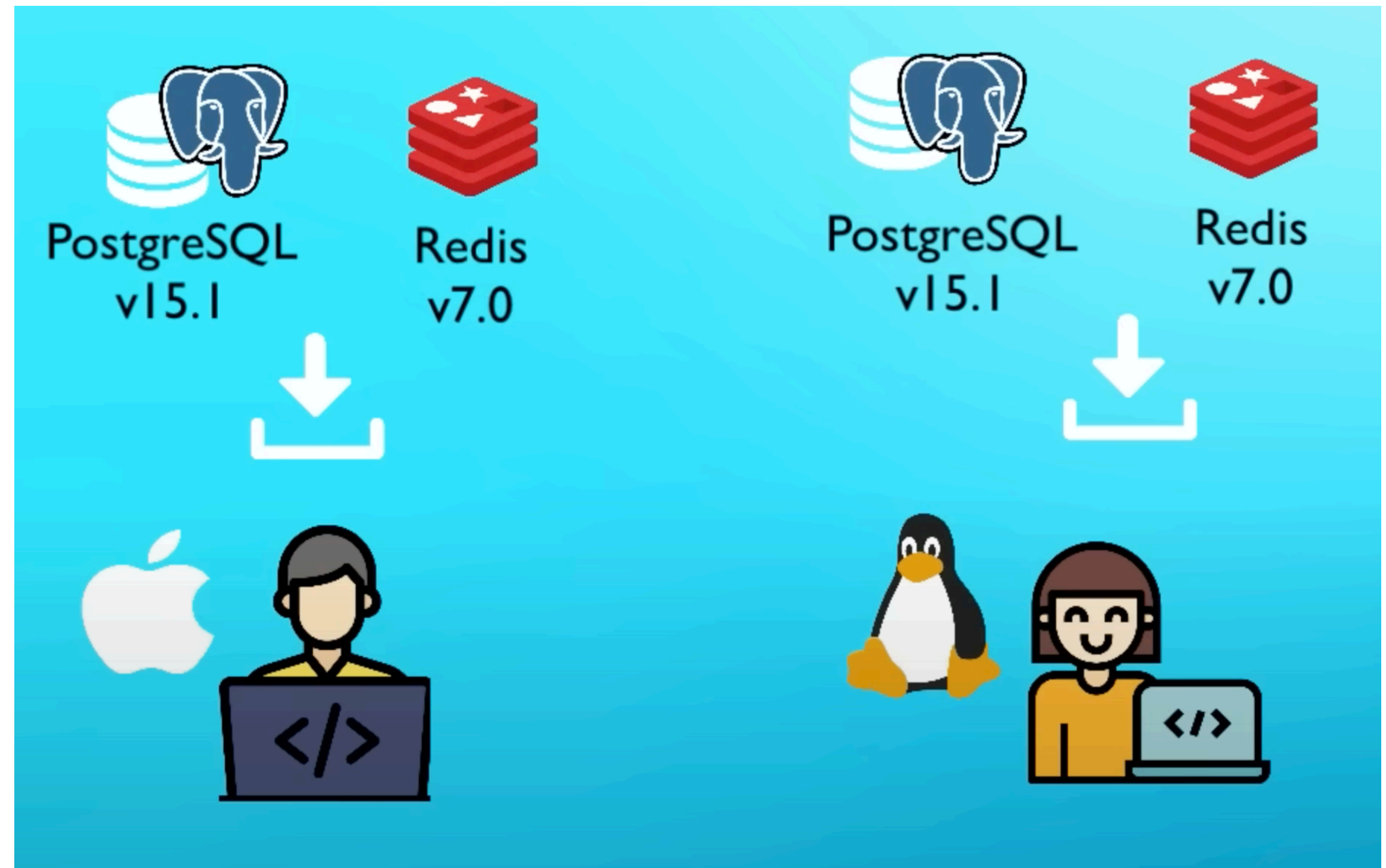
Rešitve

Open Container Initiative - OCI



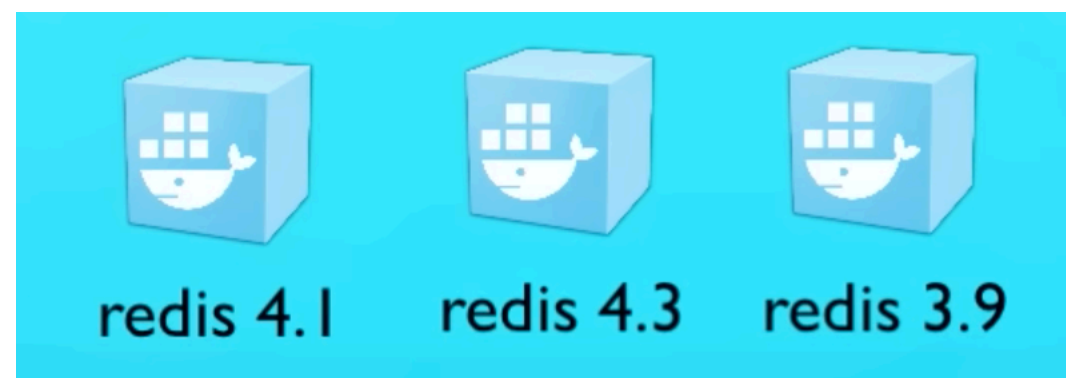
Scenarij

- Vsak mora urediti svoje okolje
- Različni OS imajo različne postopke instalacije
- Verjetnost, da kdo ima težave je 1



Scenarij

- Vsebnik vsebuje ves service
- Vso konfiguracijo
- Poenoten način zagona
- Enake ukaze ne glede na OS
- Enake ukaze za vse servise
- Ni konfliktov



`docker run postgres`

`docker run redis`

Scenarij

- Ni konfiguracijskih datotek za deploy
- Vse se zapakira v vsebnik
- Zmanjšamo verjetnost napak

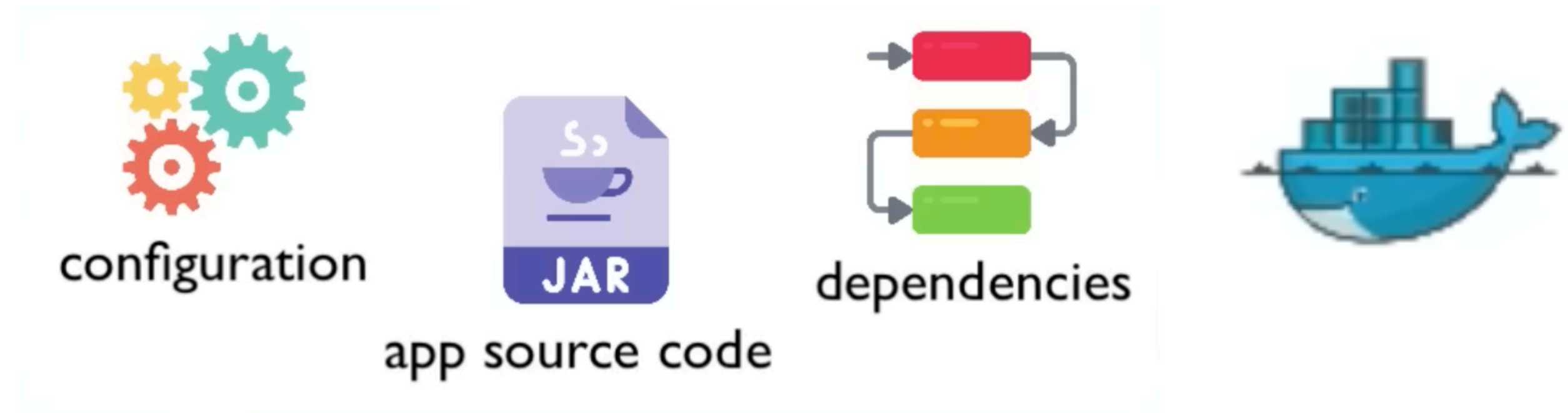


Image / Container

- Izvirno kodo, konfiguracije in podporne skripte
- Ga lahko delimo
 - ZIP / JAR
- To kar ustvarimo z Docker je Docker Image
 - Konfiguracijo
 - OS info
 - Podporo

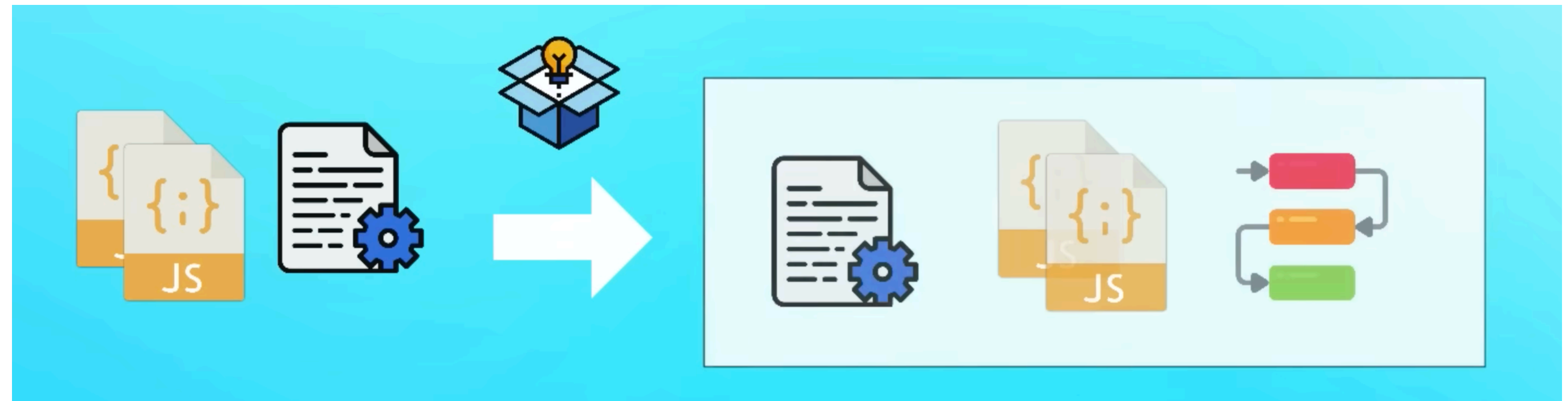


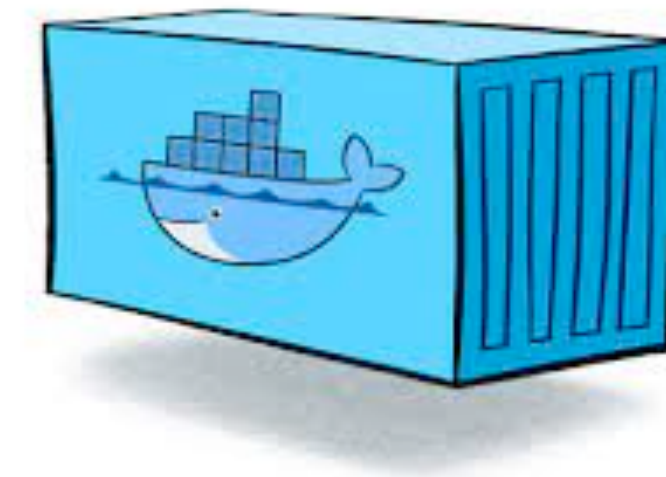
Image / Container



Aplikacija

Podporne komponente

OS Layer



- Aplikacijo moramo uporabljati
- Delujoča instance "image" je vsebnik (container)
- Takrat, ko se ustvari okolje
- Od 1 slike lahko imamo več vsebnikov

Image / Container

Seznam

pavleb@ostriga:~\$ sudo docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mediagis/nominatim	4.2	b69f27b56e5a	3 months ago	1.12GB
openrouteservice/openrouteservice	latest	8725c2989fb1	6 months ago	1.06GB
maptiler/tileserver-gl	latest	199d606f41d9	6 months ago	666MB
ubuntu	latest	7e0aa2d69a15	2 years ago	72.7MB
hello-world	latest	fce289e99eb9	4 years ago	1.84kB

Image / Container

Vsebniki

```
pavleb@ostriga:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS	PORTS	NAMES	
bc3be4b0ba45 ago Up 3 months	openrouteservice/openrouteservice:latest 0.0.0.0:9998->8080/tcp, :::9998->8080/tcp	"/bin/bash /ors-core..." ors-app	3 months
c1c0221a4ae5 ago Up 3 months	mediagis/nominatim:4.2 5432/tcp, 0.0.0.0:9999->8080/tcp, :::9999->8080/tcp	"/app/start.sh" nominatim	3 months

Image

Kje jih dobimo?

- Uradni repo
- Docker hub



redis



DOCKER OFFICIAL IMAGE · 📄 1B+ · ☆ 10K+

Redis is an open source key-value store that functions as a data structure server.

```
docker pull redis
```



Image



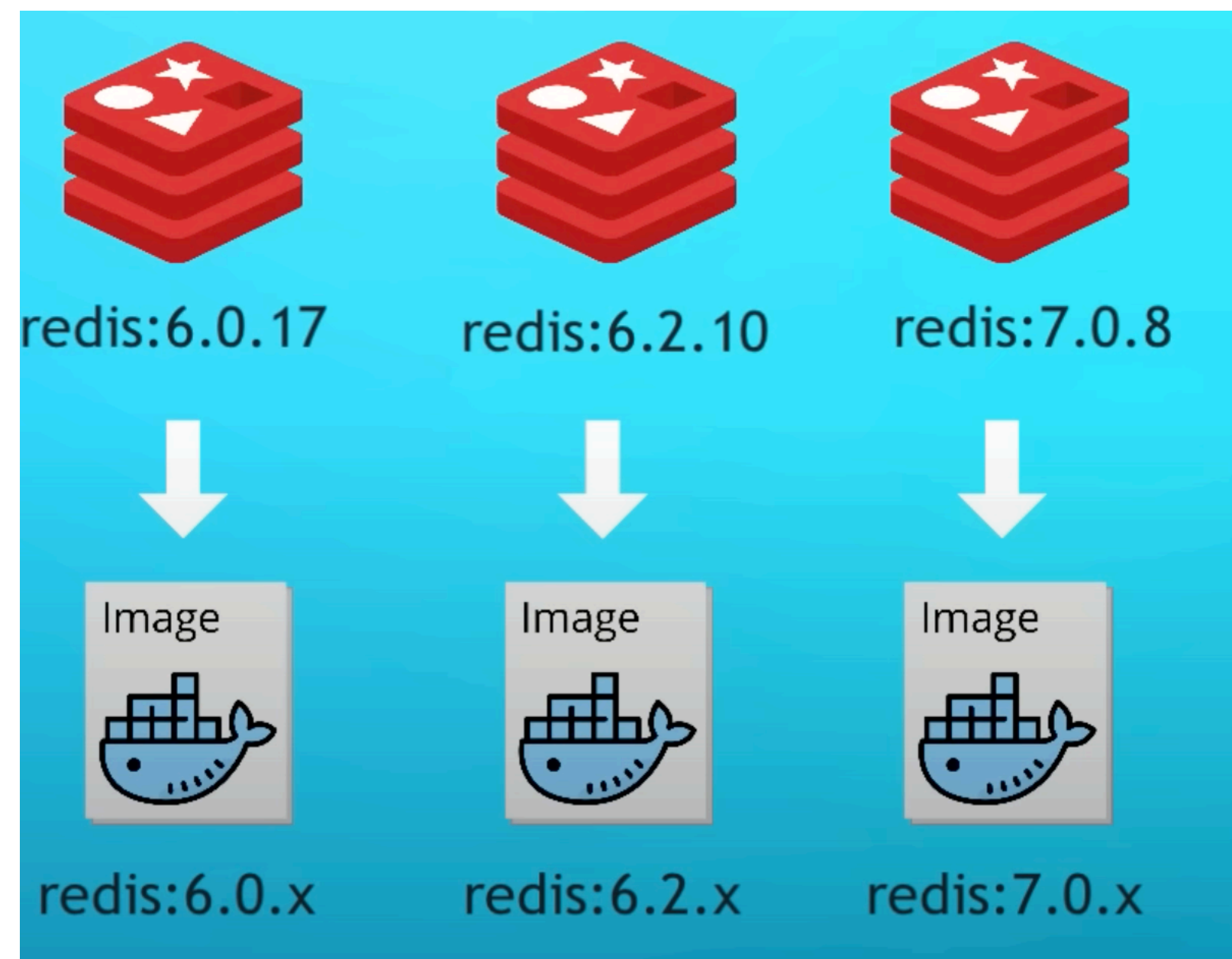
redis

DOCKER OFFICIAL IMAGE

• 1B+ • 10K+

Redis is an open source key-value store that functions as a data structure server.

```
docker pull redis
```



```
docker pull {name}:{tag}
```

Container

`docker run {name}:{tag}`

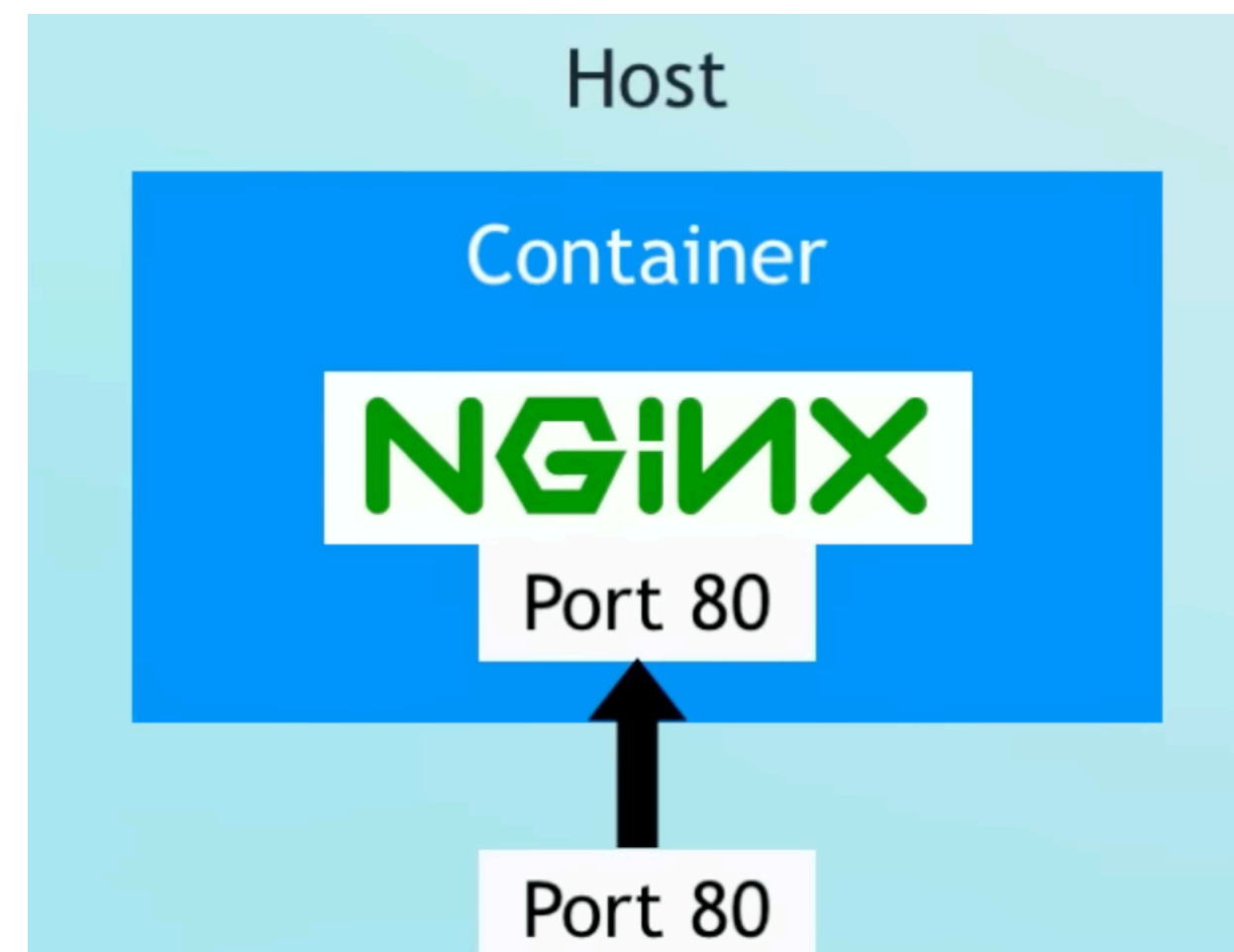
`docker ps`

`docker -d run {name}:{tag}` Detach

`docker logs {containerid}` Pogledati logs

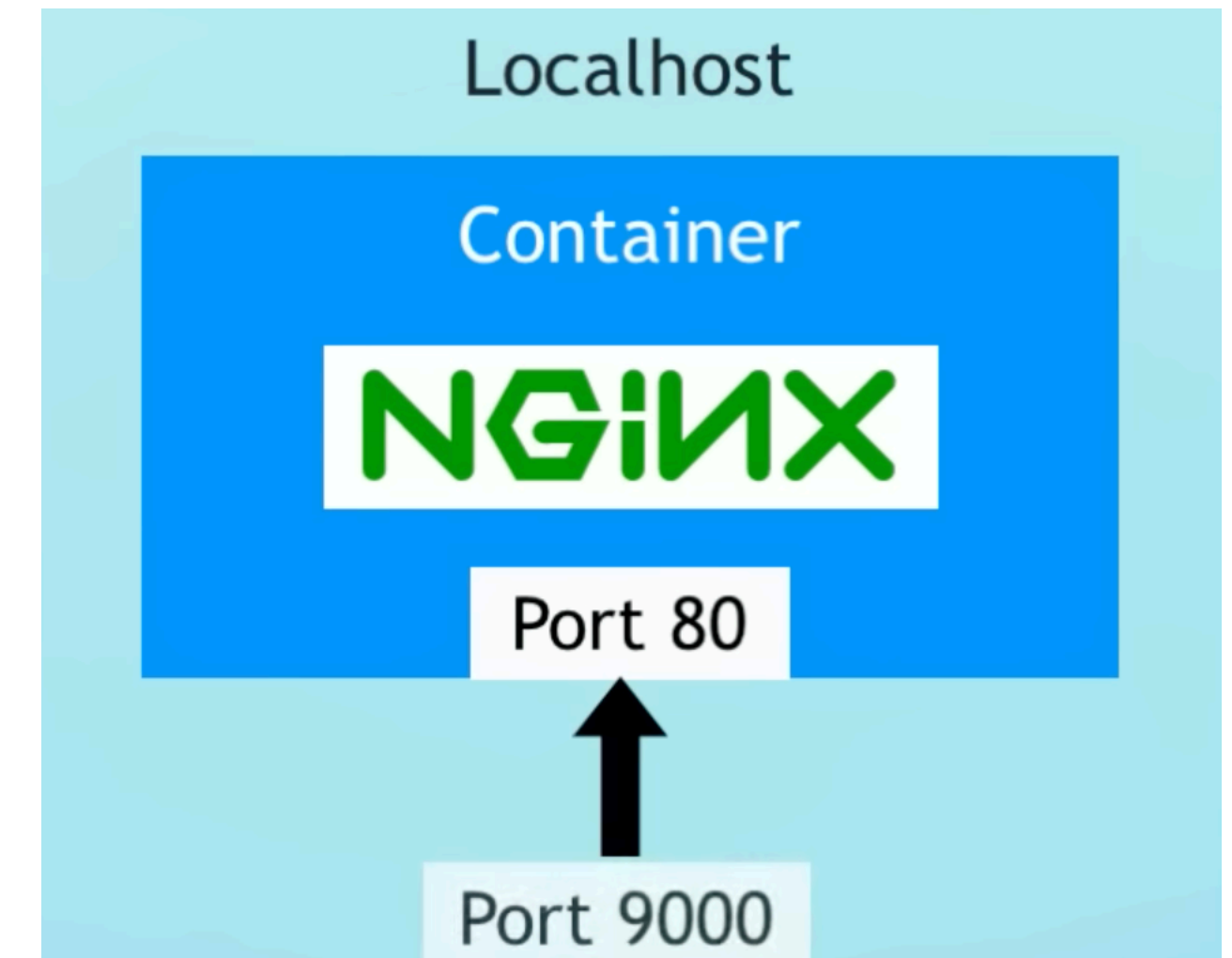
Container

- Vsebnik dela v "izoliranem" omrežju Docker
 - Lahko imamo več vsebnikov, ki uporabljajo iste porte
- Da lahko dostopamo
 - Expose ports
 - Port binding



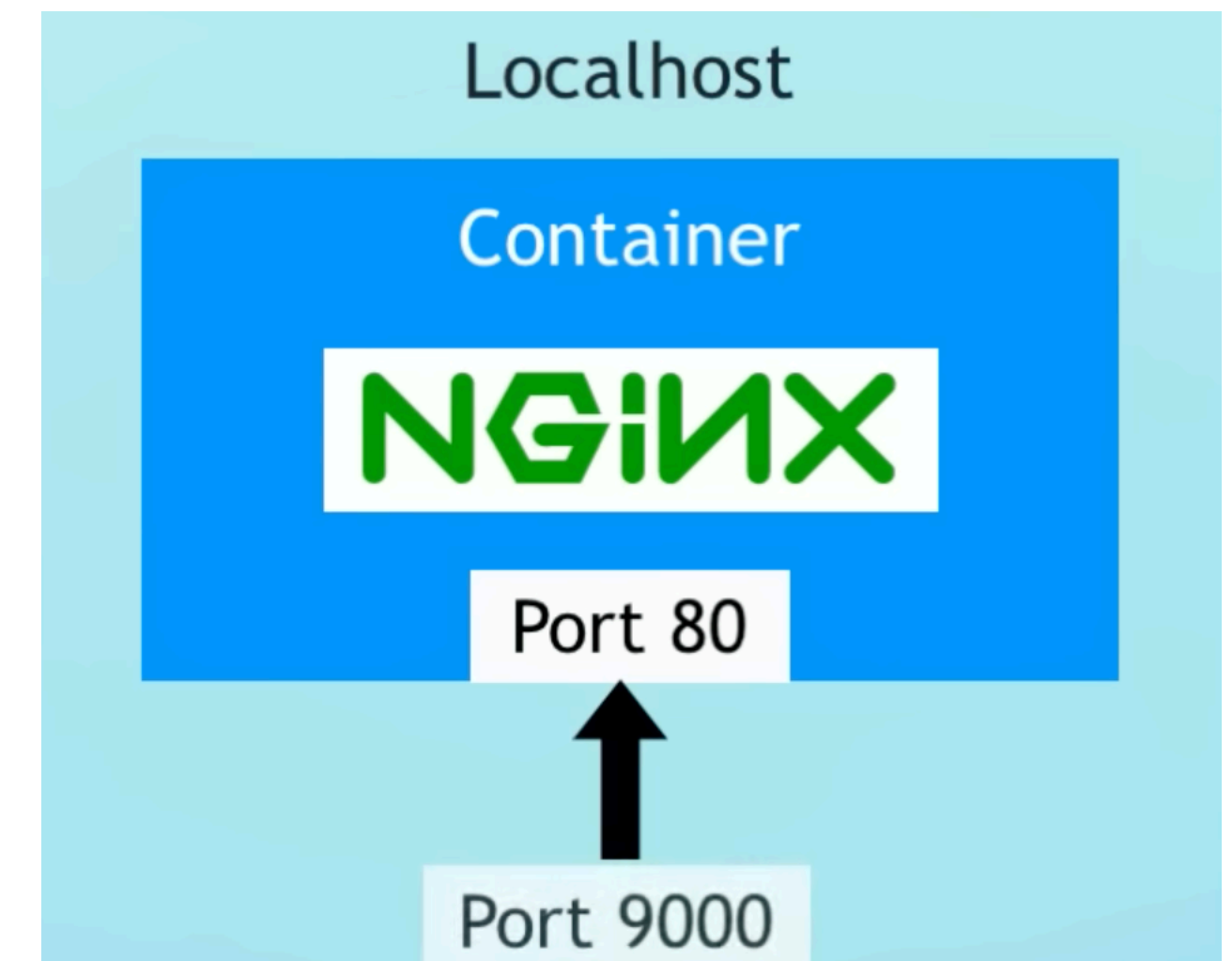
Container

- `docker stop {container_id}`
- `docker run -d -p 9000:80 {name}:{tag}`
 - Port binding



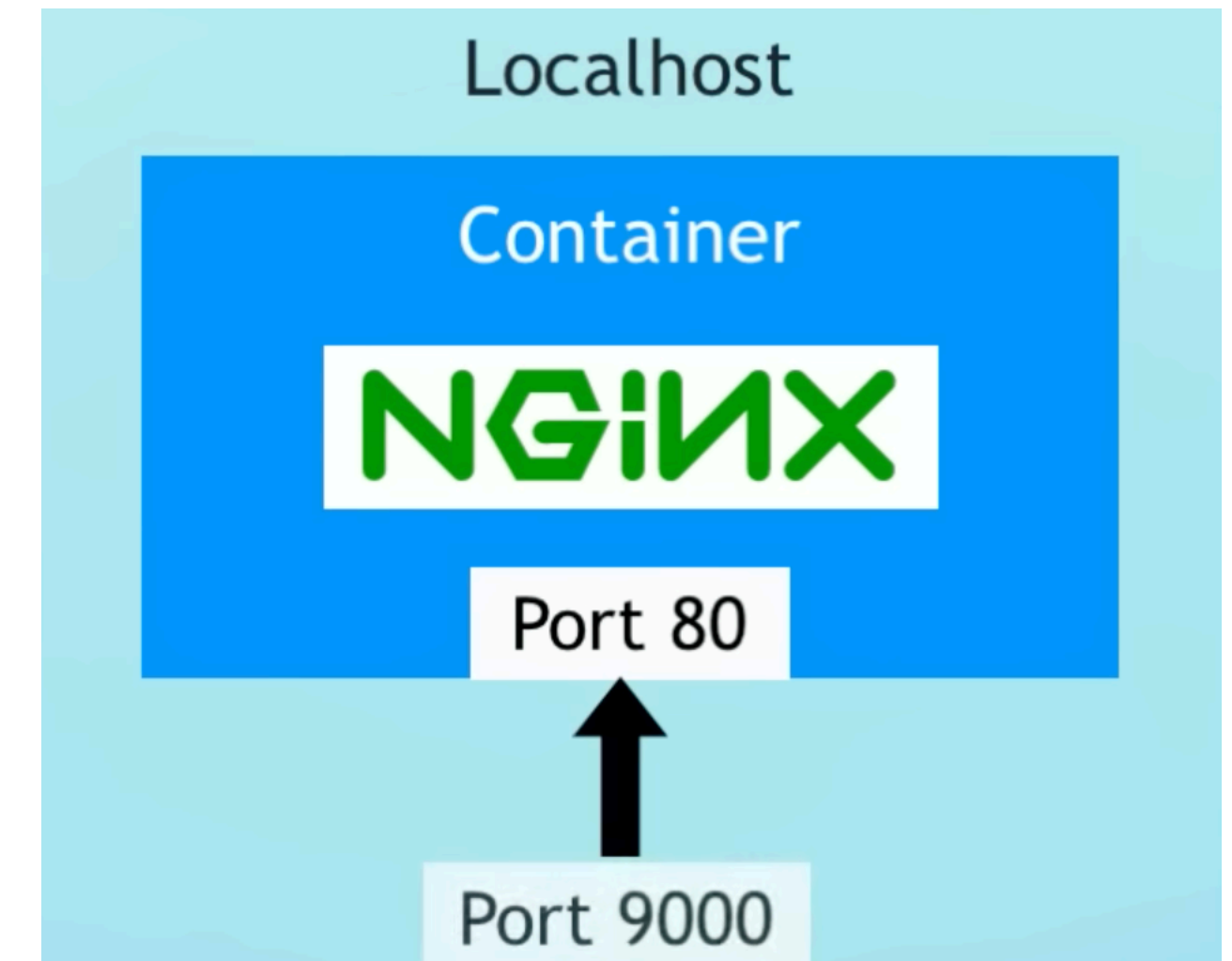
Start/Stop

- `docker run -d -p 9000:80 {name}:{tag}`
 - Vedno naredi nov container
 - Ne uporablja že nastavljenega
- `docker ps`
 - Pokaže delujočih
- `docker ps -a`
- `docker start {container id}`



Start/Stop

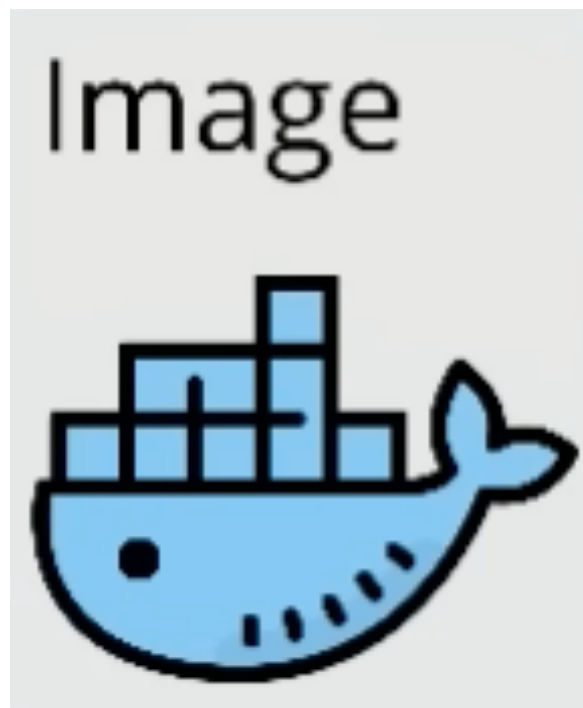
- `docker run --name IME-d -p 9000:80 {name}: {tag}`
- `docker stop IME`
- `docker start IME`



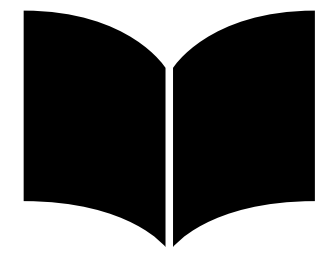
Custom vsebnik

Razvoj končan

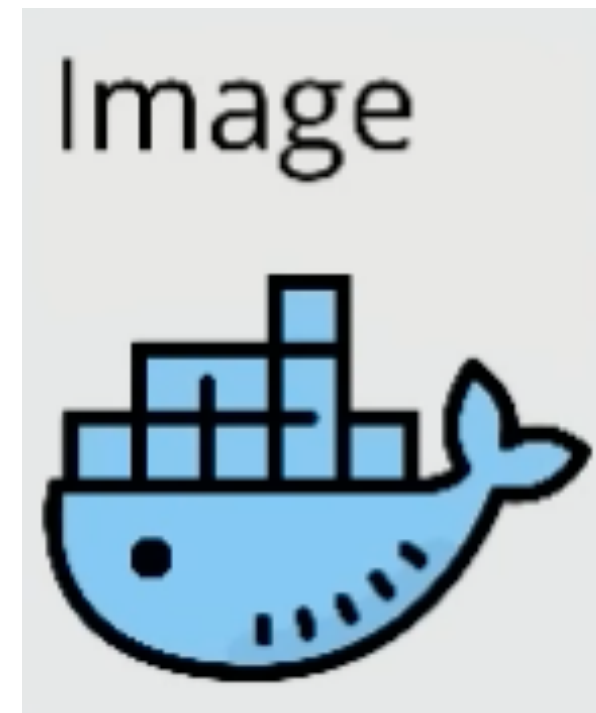
Deployment



Dockerfile



Dockerfile

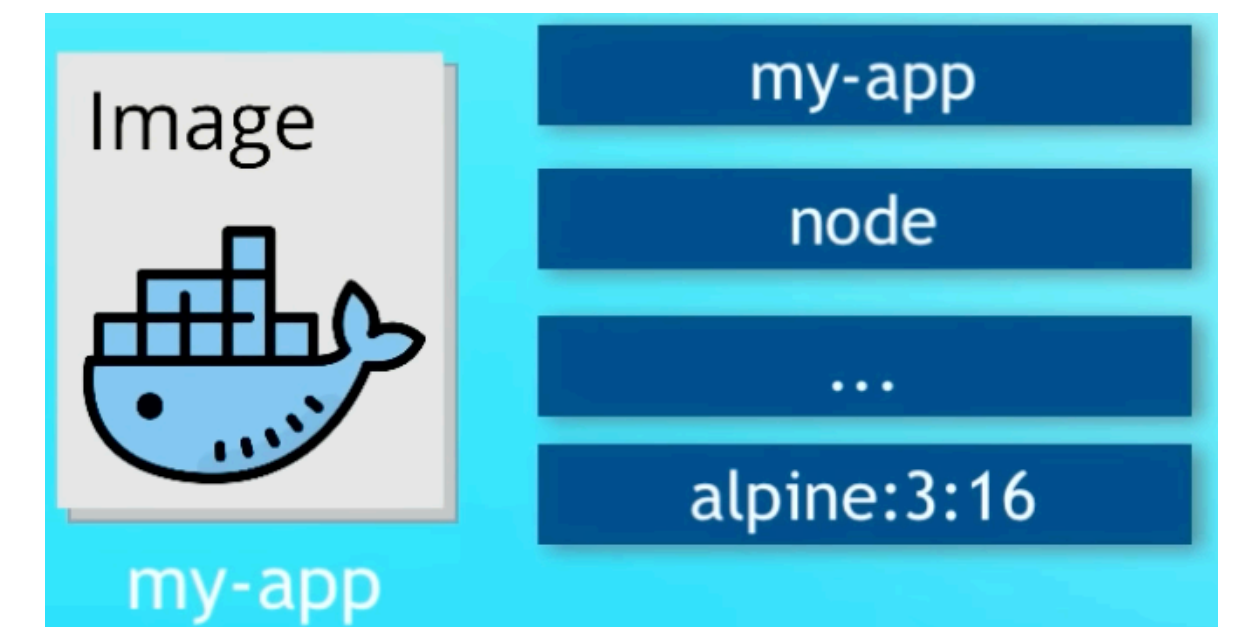
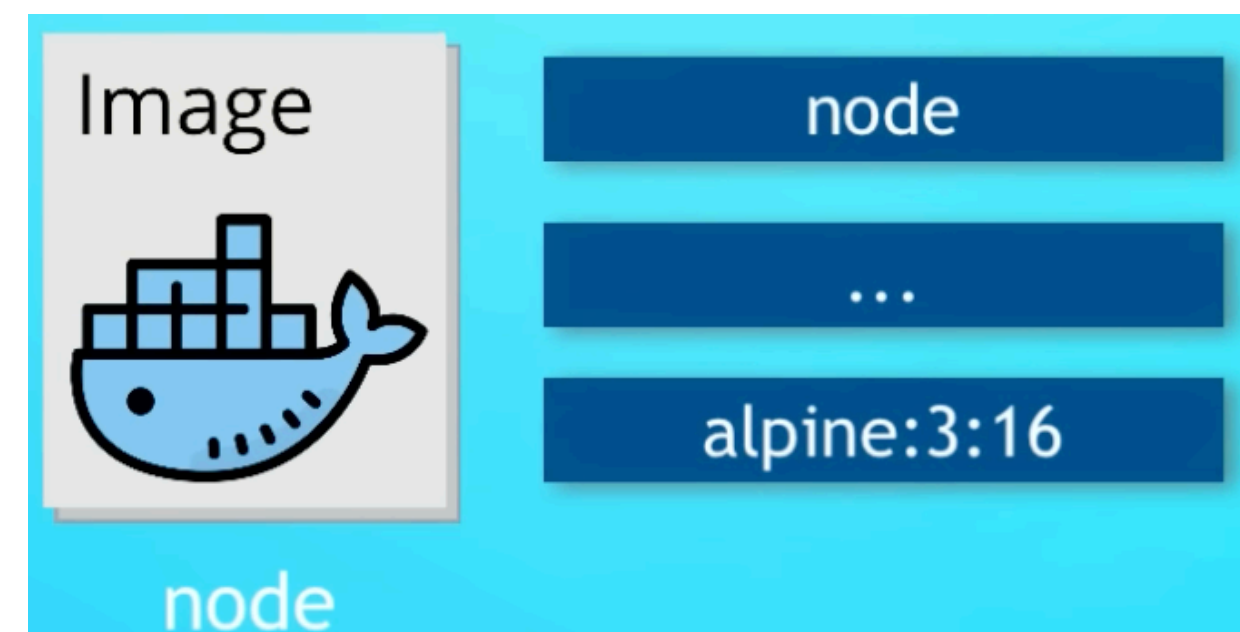


Docker
image

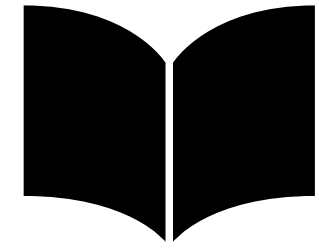


Docker
container

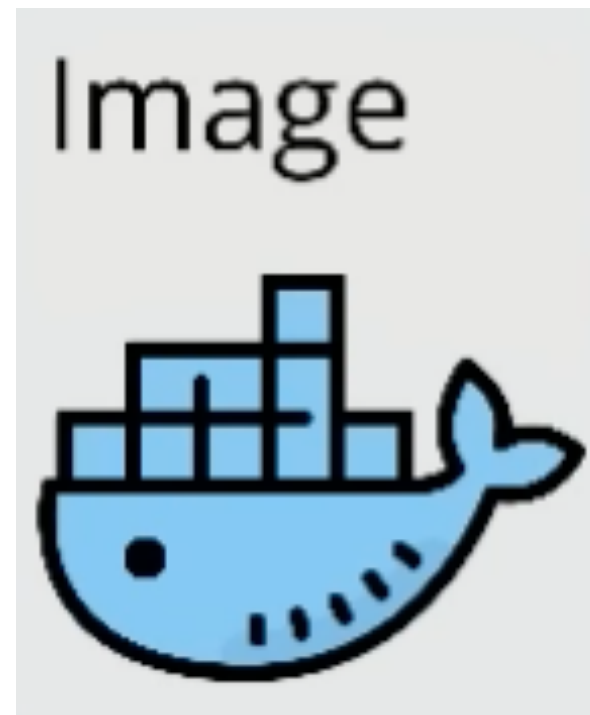
- Osnova - Base FROM
- Običajno Lightweight Linux



Dockerfile



Dockerfile



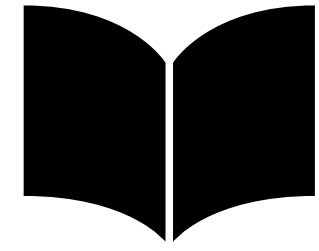
Docker
image



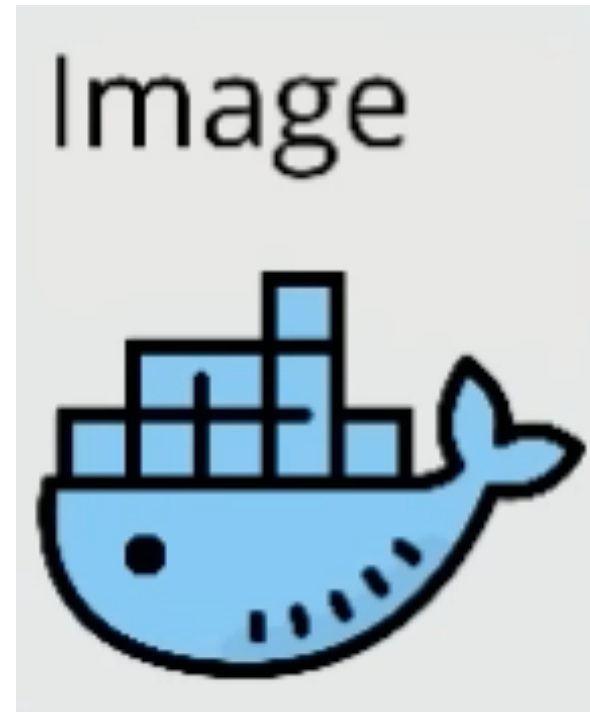
Docker
container

```
FROM node:19-alpine
```

Dockerfile



Dockerfile



Docker
image

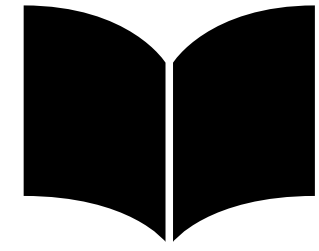


Docker
container

```
FROM node:19-alpine
```

```
# Install app dependencies  
RUN npm install
```

Dockerfile



Dockerfile



Docker
image

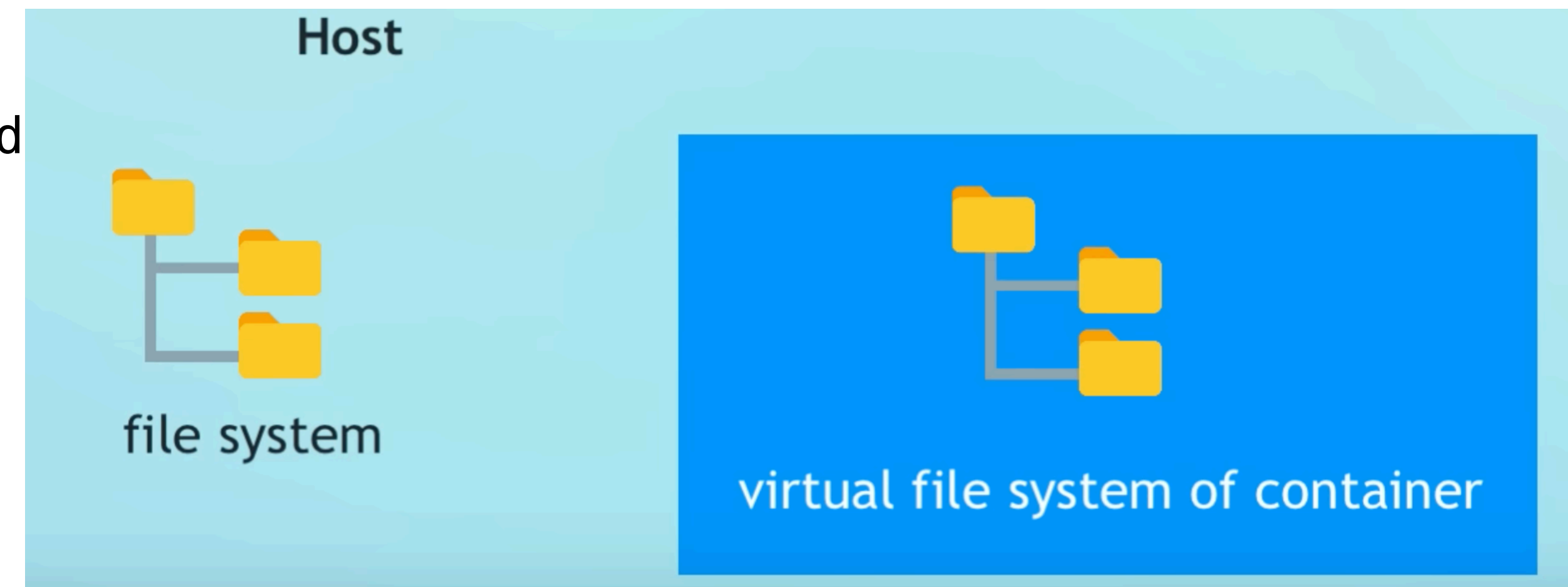


Docker
container

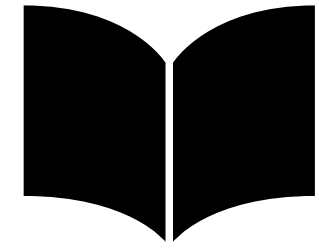
```
FROM node:19-alpine
```

```
# Copy package.json, wildcard used so both  
package.json AND package-lock.json are copied  
# slash '/' at the end of app is important,  
so it created an app directory, otherwise  
you'll get an error  
COPY package*.json /usr/app/
```

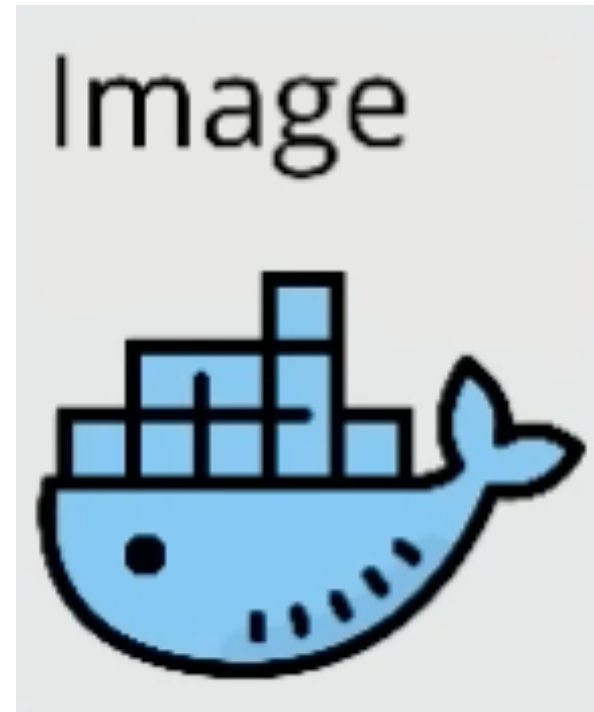
```
# Install app dependencies  
RUN npm install
```



Dockerfile



Dockerfile



Docker
image



Docker
container

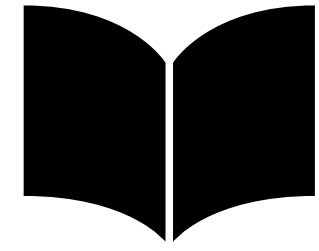
```
FROM node:19-alpine
```

```
# Copy package.json, wildcard used so both package.json AND package-lock.json are copied  
# slash '/' at the end of app is important, so it created an app directory, otherwise  
you'll get an error  
COPY package*.json /usr/app/
```

```
# Copy app files from src directory  
COPY src /usr/app/
```

```
# Install app dependencies  
RUN npm install
```

Dockerfile



Dockerfile



Docker
image



Docker
container

```
FROM node:19-alpine
```

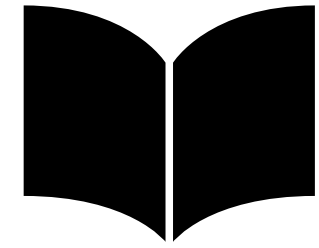
```
# Copy package.json, wildcard used so both package.json AND package-lock.json are copied  
# slash '/' at the end of app is important, so it created an app directory, otherwise  
you'll get an error  
COPY package*.json /usr/app/
```

```
# Copy app files from src directory  
COPY src /usr/app/
```

```
# Create app directory & set default dir so that next commands executes in /usr/app dir,  
liked cd-ing into /usr/app to execute npm install  
WORKDIR /usr/app
```

```
# Install app dependencies  
RUN npm install
```

Dockerfile



Dockerfile

```
FROM node:19-alpine
```

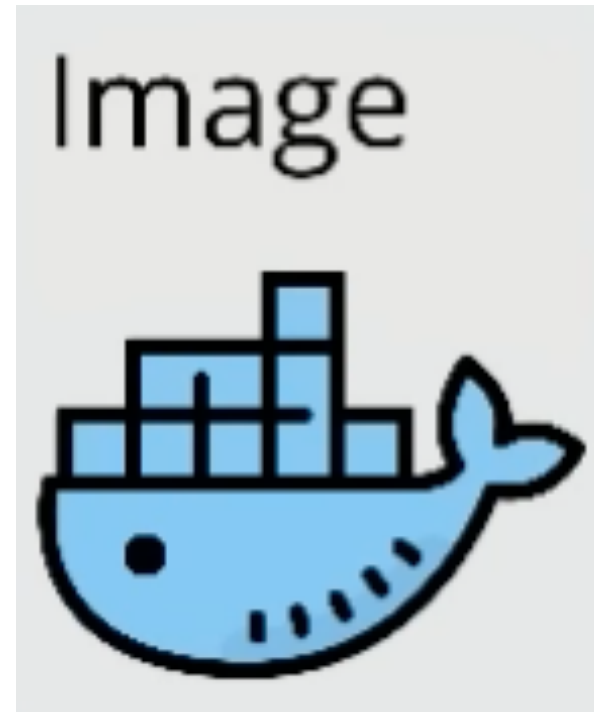
```
# Copy package.json, wildcard used so both package.json AND package-lock.json are copied  
# slash '/' at the end of app is important, so it created an app directory, otherwise  
you'll get an error  
COPY package*.json /usr/app/
```

```
# Copy app files from src directory  
COPY src /usr/app/
```

```
# Create app directory & set default dir so that next commands executes in /usr/app dir,  
liked cd-ing into /usr/app to execute npm install  
WORKDIR /usr/app
```

```
# Install app dependencies  
RUN npm install
```

```
# Start the application  
CMD ["node", "server.js"]
```

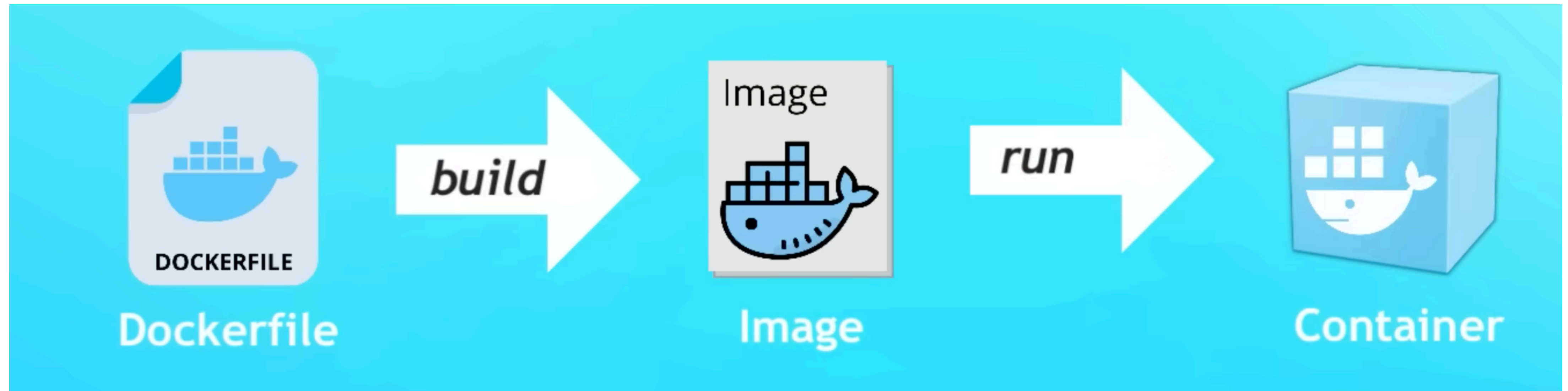


Docker
image



Docker
container

Custom vsebnik



```
docker build -t my_app:1.0 .
```

```
docker images
```

Multicontainer docker

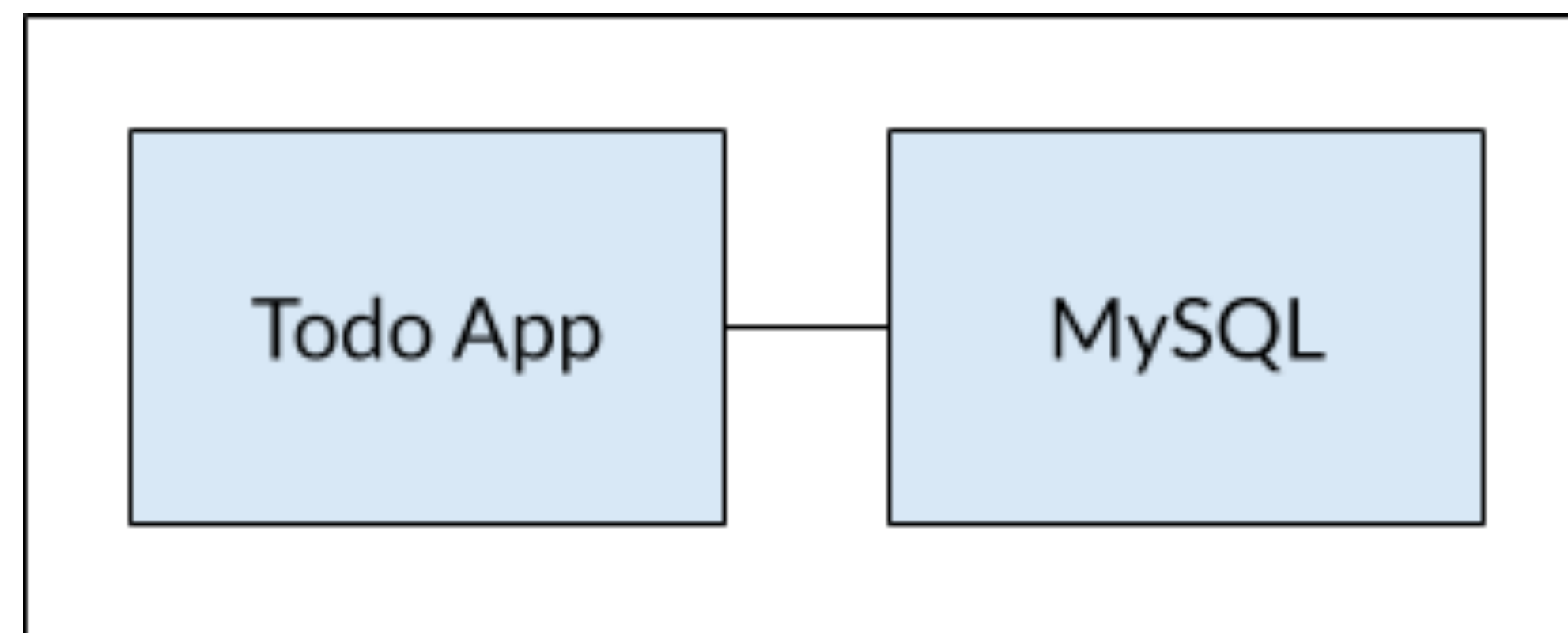
Vsebnik lahko postavite v omrežje na dva načina:

Pri zagonu vsebnika dodelite omrežje.

Povežite vsebnik, ki že deluje, v omrežje.

```
docker network create todo-app
```

```
docker run -d \  
  --network todo-app --network-alias mysql \  
  -v todo-mysql-data:/var/lib/mysql \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  -e MYSQL_DATABASE=todos \  
  mysql:8.0
```

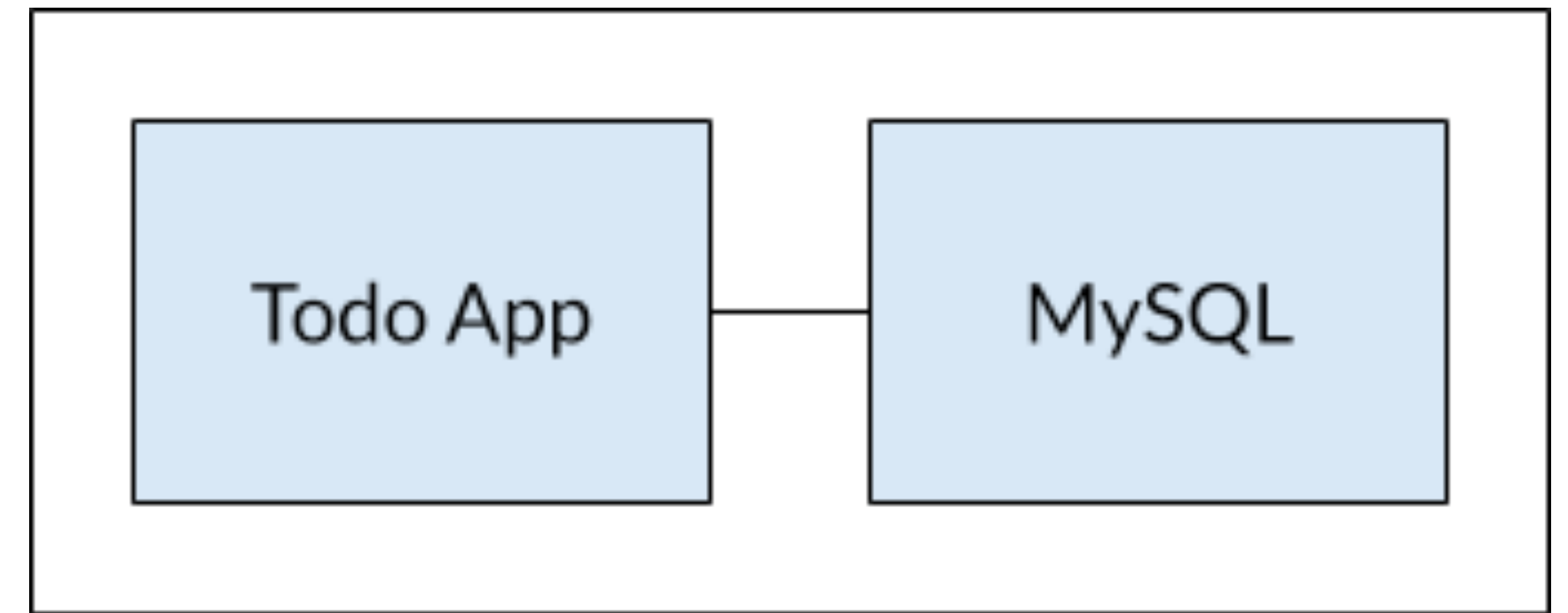


Zunanji disk mount

Vsebnik lahko postavite v omrežje na dva načina:

Pri zagonu vsebnika dodelite omrežje.

Povežite vsebnik, ki že deluje, v omrežje.

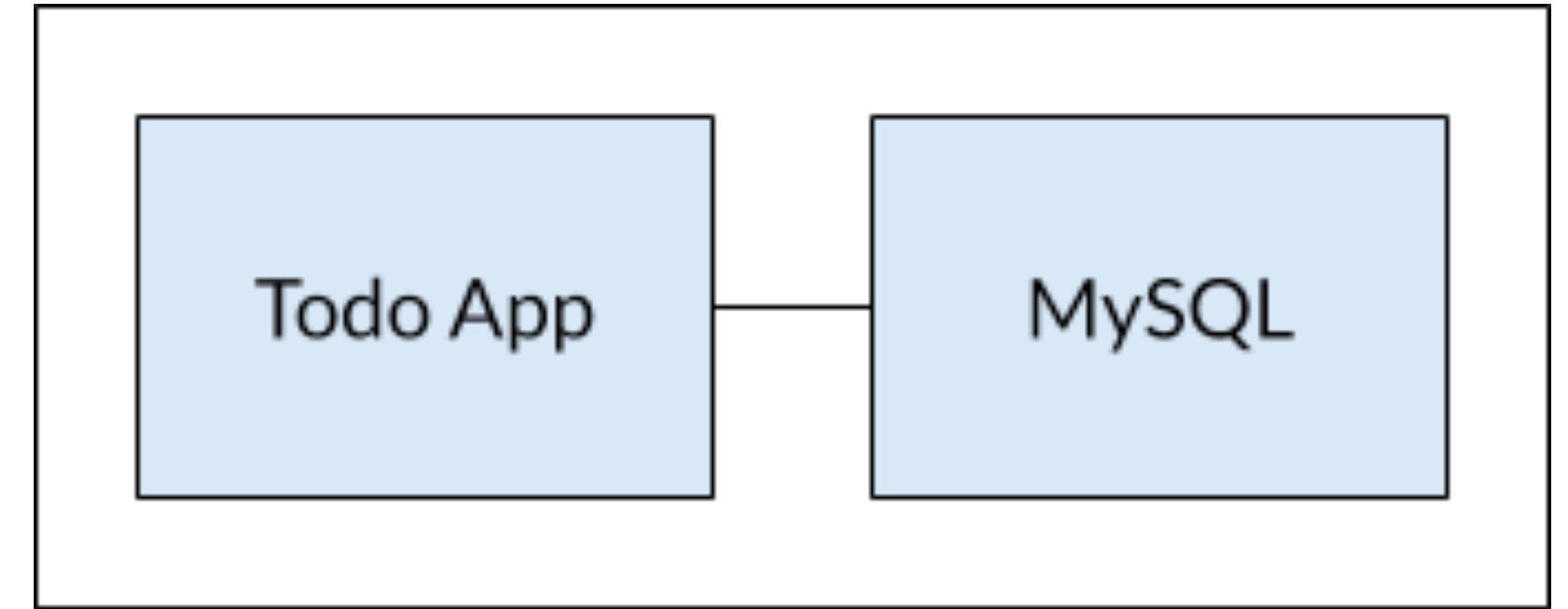


```
docker network create todo-app
```

```
docker run -d \
  --network todo-app --network-alias mysql \
  -v todo-mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=todos \
  mysql:8.0
```

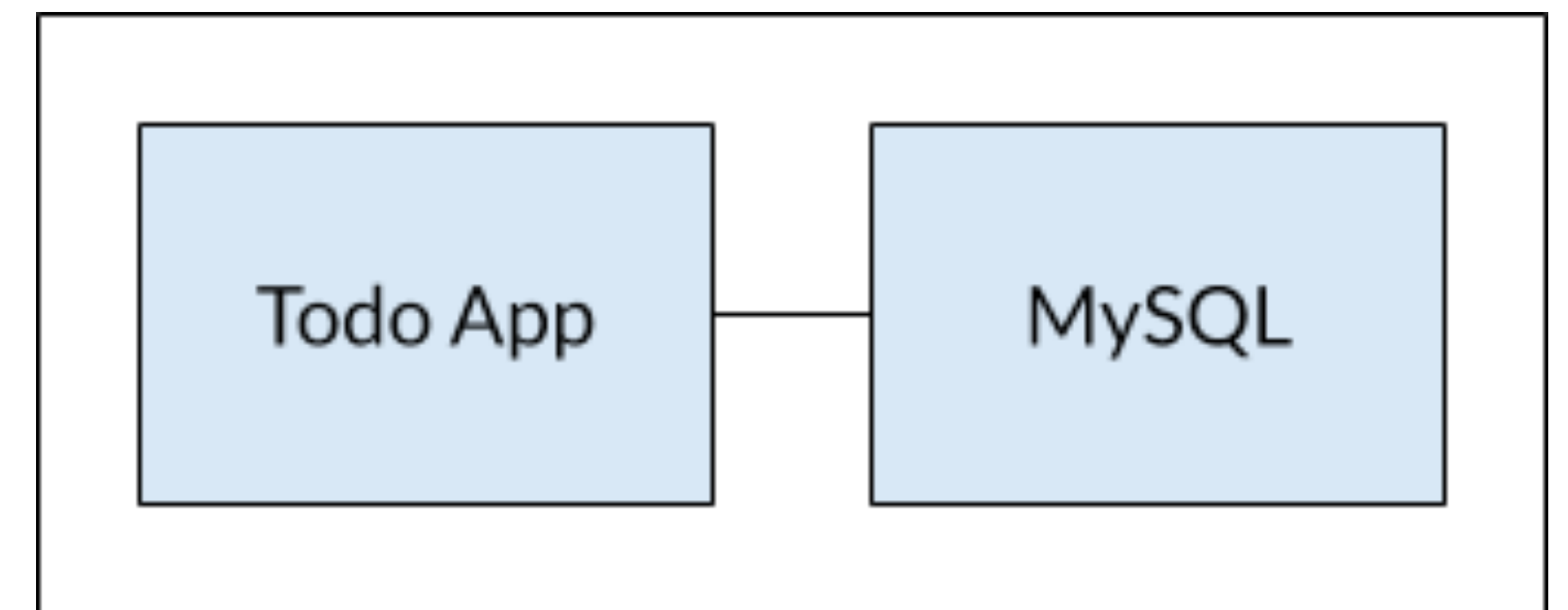
Docker compose

```
docker run -dp 3000:3000 \
  -w /app -v "$(pwd):/app" \
  --network todo-app \
  -e MYSQL_HOST=mysql \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=secret \
  -e MYSQL_DB=todos \
  node:18-alpine \
  sh -c "yarn install && yarn run dev"
```



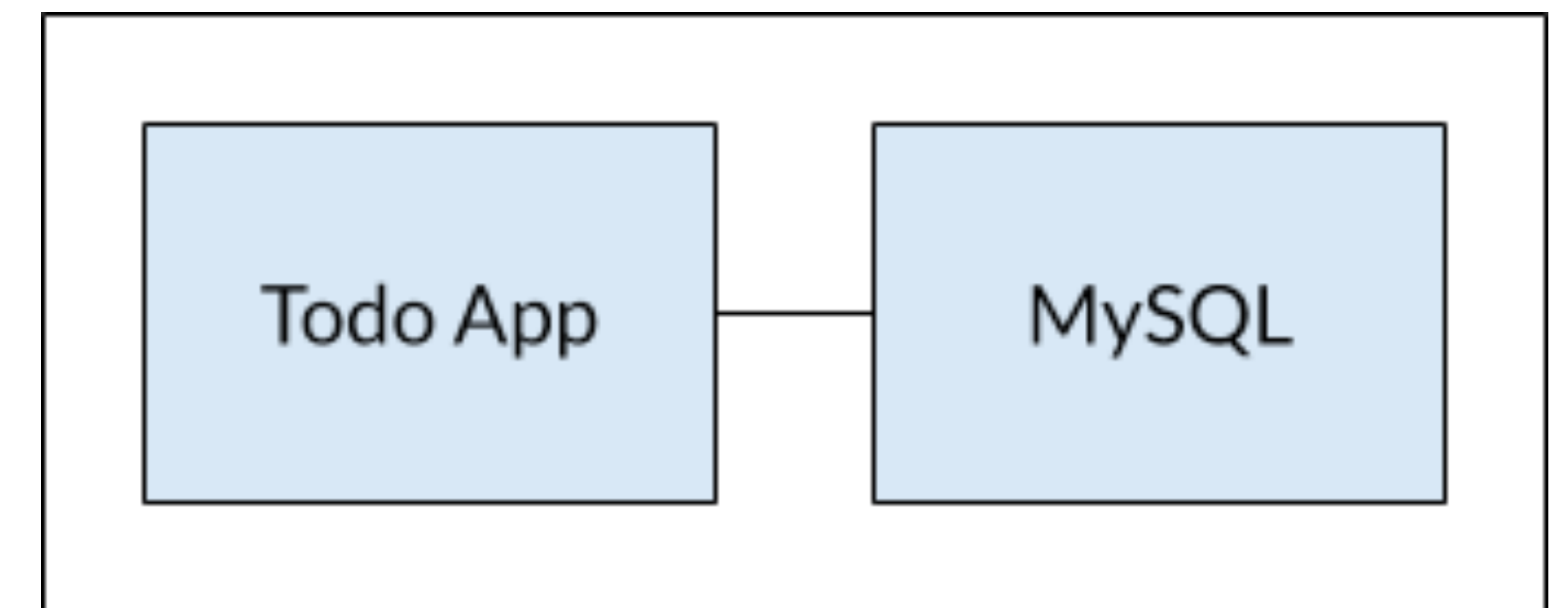
Docker compose

```
services:
  app:
    image: node:18-alpine
    command: sh -c "yarn install && yarn run dev"
    ports:
      - 3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos
```



Docker compose

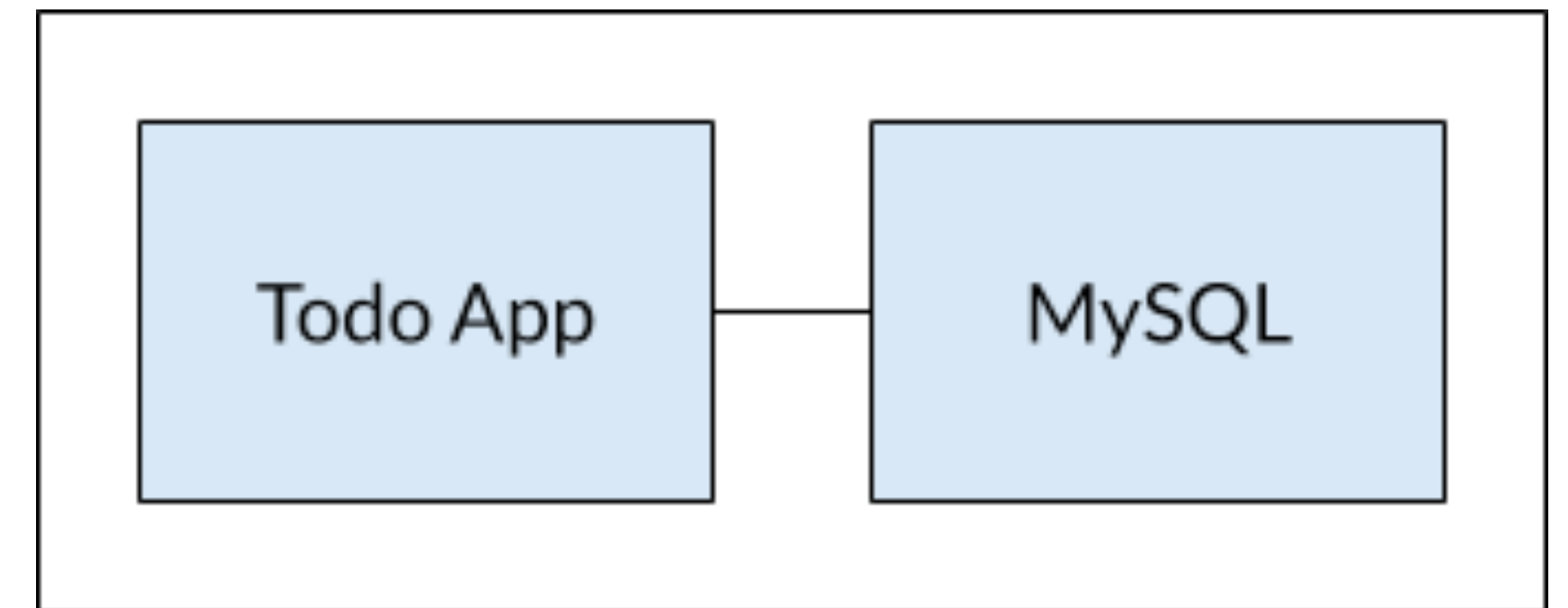
```
docker run -d \  
  --network todo-app --network-alias mysql \  
  -v todo-mysql-data:/var/lib/mysql \  
  -e MYSQL_ROOT_PASSWORD=secret \  
  -e MYSQL_DATABASE=todos \  
mysql:8.0
```



Docker compose

```
services:
  app:
    # The app service definition
  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos

volumes:
  todo-mysql-data:
```



Prostor

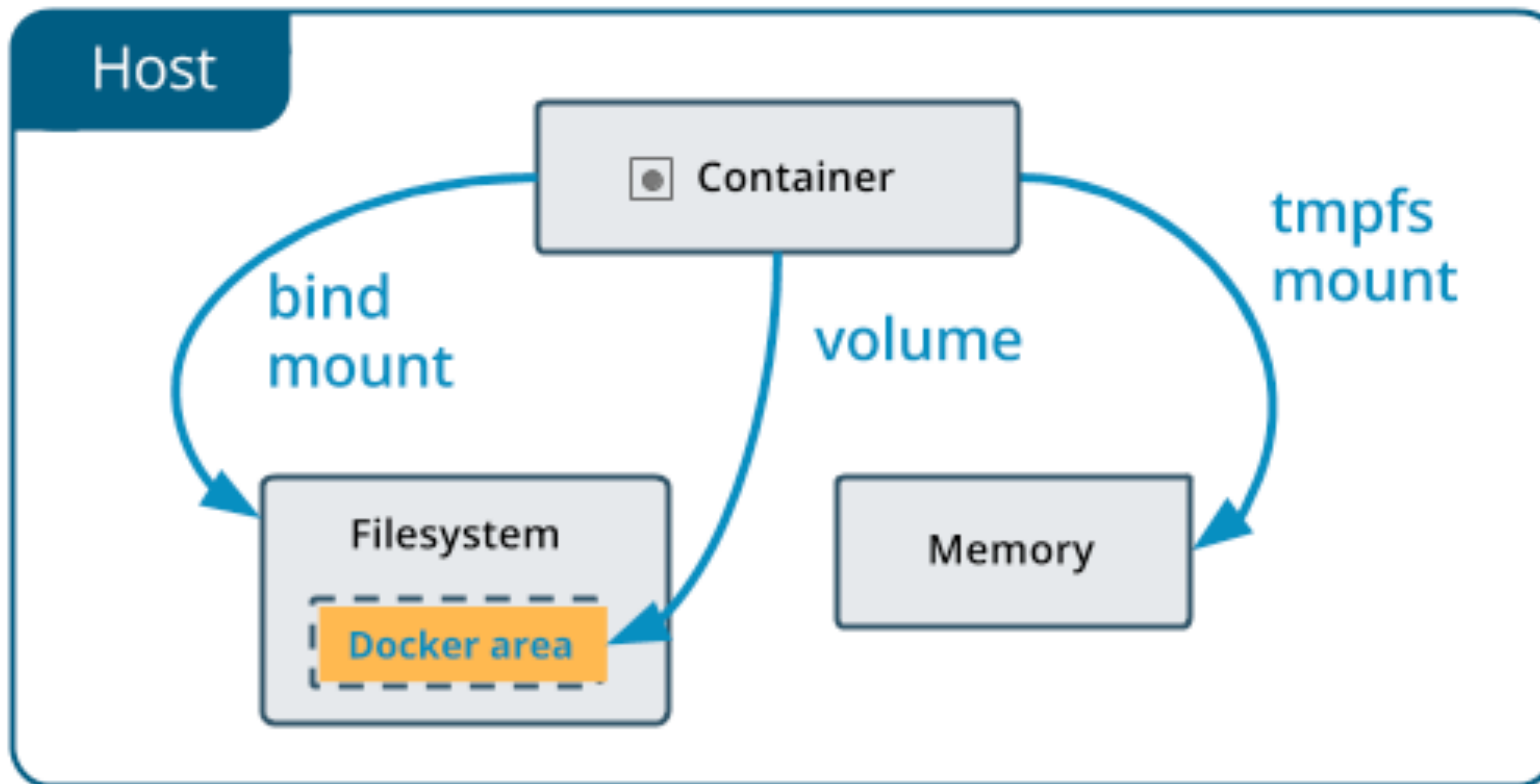
```
docker volume create todo-db
```

```
docker run -dp 3000:3000 --mount type=volume,src=todo-db,target=/etc/todos  
getting-started
```

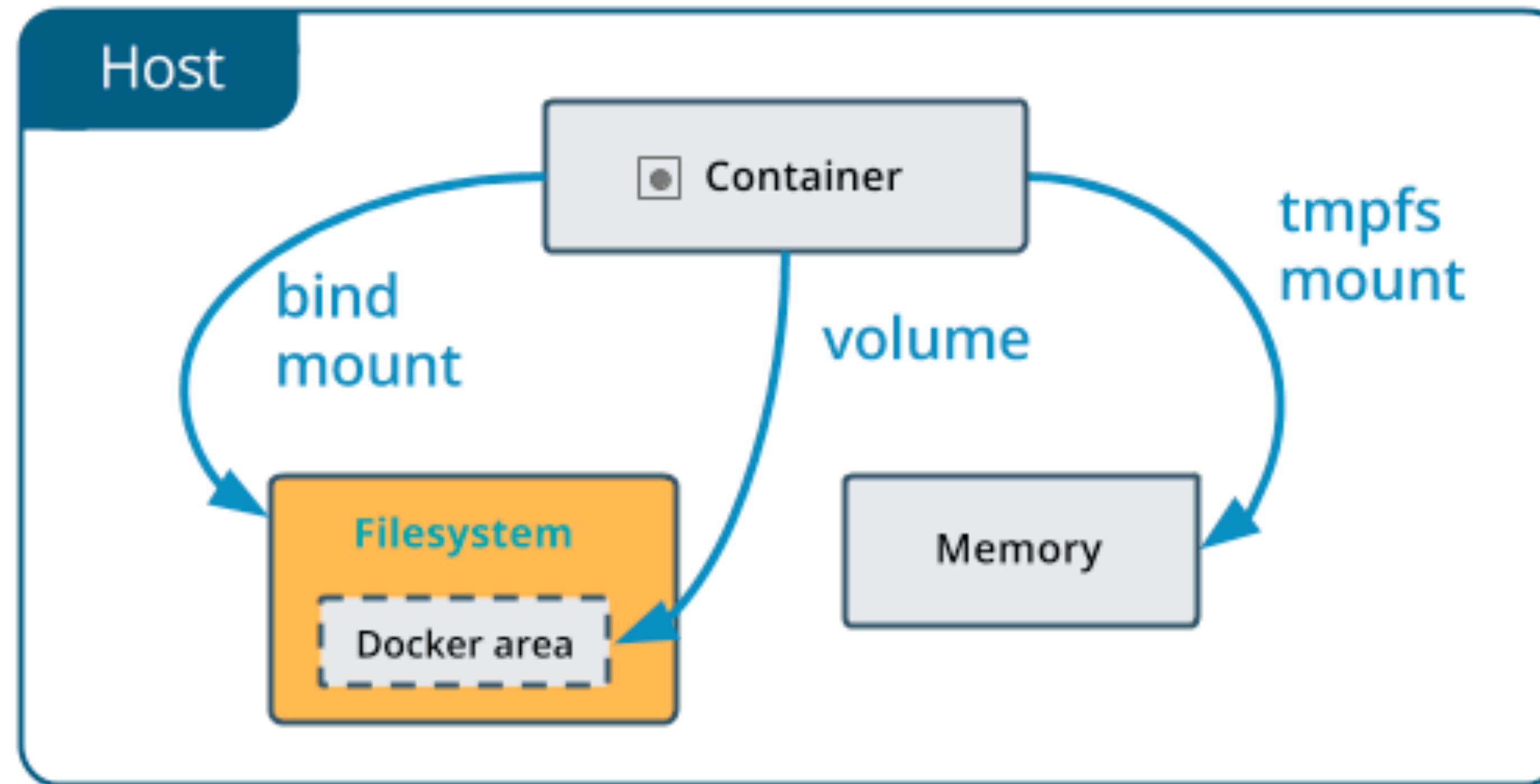
```
docker volume inspect todo-db
```

```
[  
  {  
    "CreatedAt": "2019-09-26T02:18:36Z",  
    "Driver": "local",  
    "Labels": {},  
    "Mountpoint": "/var/lib/docker/volumes/todo-db/_data",  
    "Name": "todo-db",  
    "Options": {},  
    "Scope": "local"  
  }  
]
```


Prostor



Bind mount



Prostor

```
docker run -d \
  -it \
  --name devtest \
  --mount type=bind,source="$(pwd)"/target,target=/app \
  nginx:latest
```

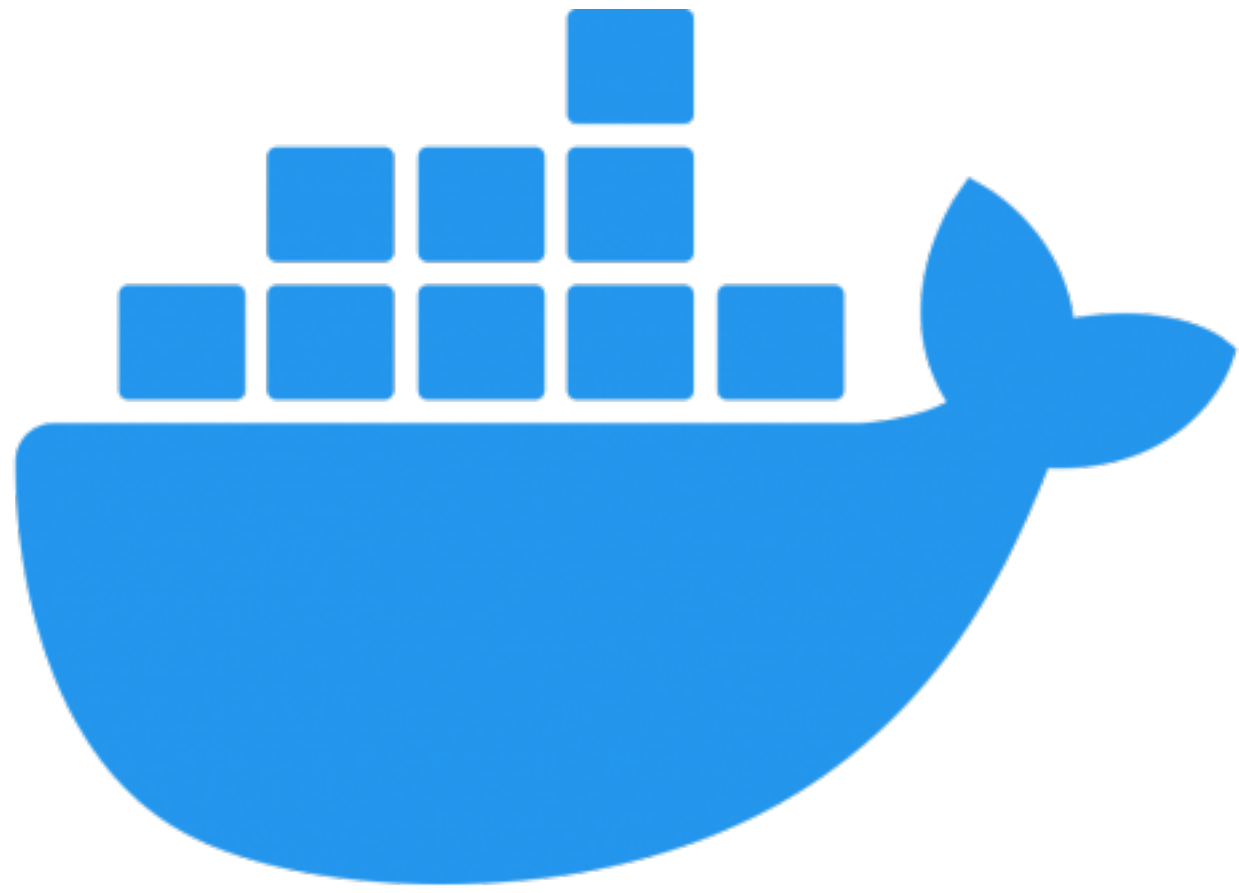
```
docker run -d \
  -it \
  --name devtest \
  -v "$(pwd)"/target:/app \
  nginx:latest
```

```
docker inspect devtest
"Mounts": [
  {
    "Type": "bind",
    "Source": "/tmp/source/target",
    "Destination": "/app",
    "Mode": "",
    "RW": true,
    "Propagation": "rprivate"
  }
],
```

Mount

	Named volumes	Bind mounts
Host location	Docker chooses	You decide
Mount example (using <code>--mount</code>)	<code>type=volume,src=my-volume,target=/usr/local/data</code>	<code>type=bind,src=/path/to/data,target=/usr/local/data</code>
Populates new volume with container contents	Yes	No
Supports Volume Drivers	Yes	No

Singularity



docker®



Singularity

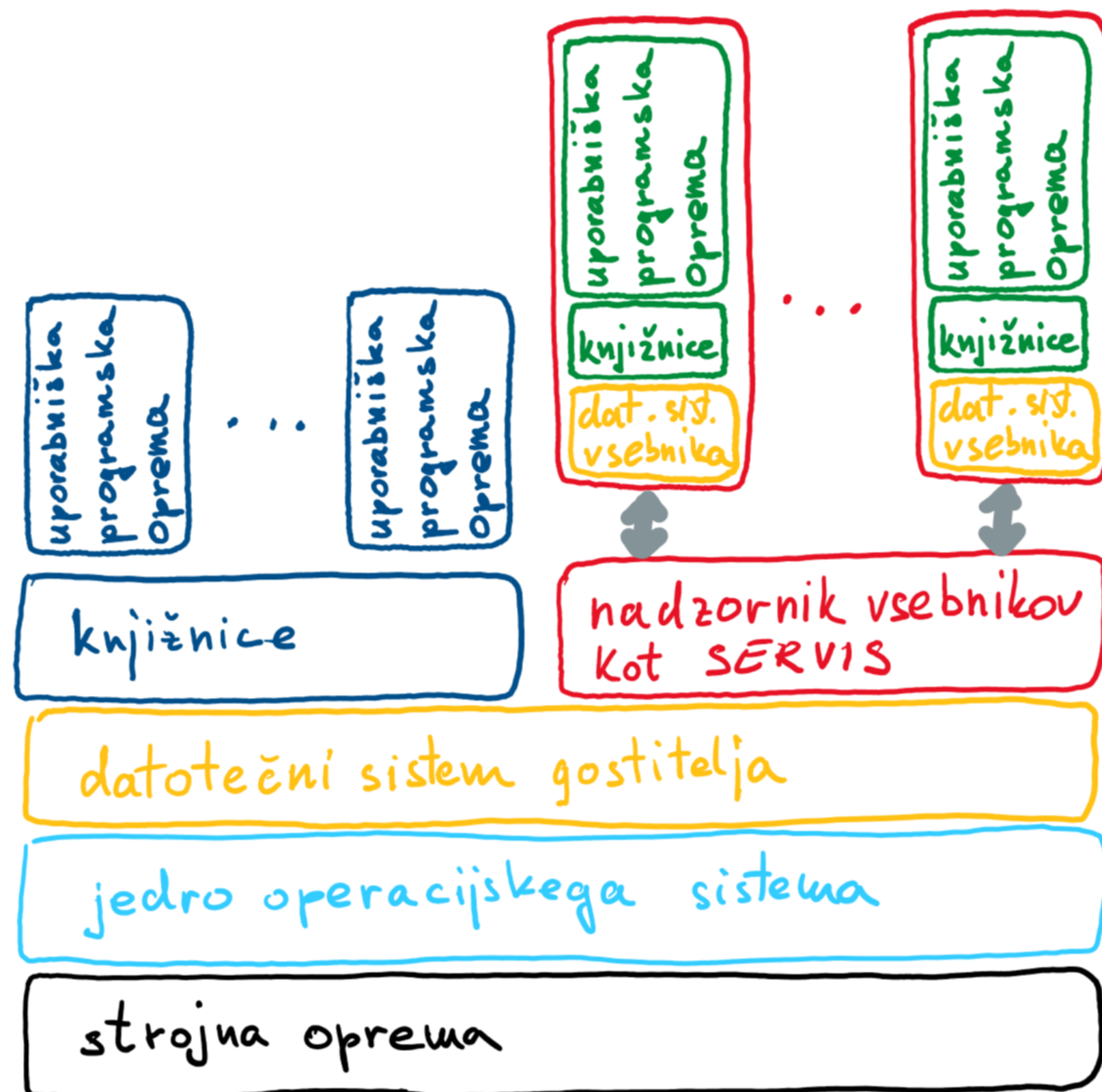
- Vsebniki Docker - nadzornik vsebnikov teče kot servis, ki za zagon vsebnika in mnoge operacije z njimi zahteva več pravic, kot jih imajo običajni uporabniki.

Prednosti ogrodja Singularity

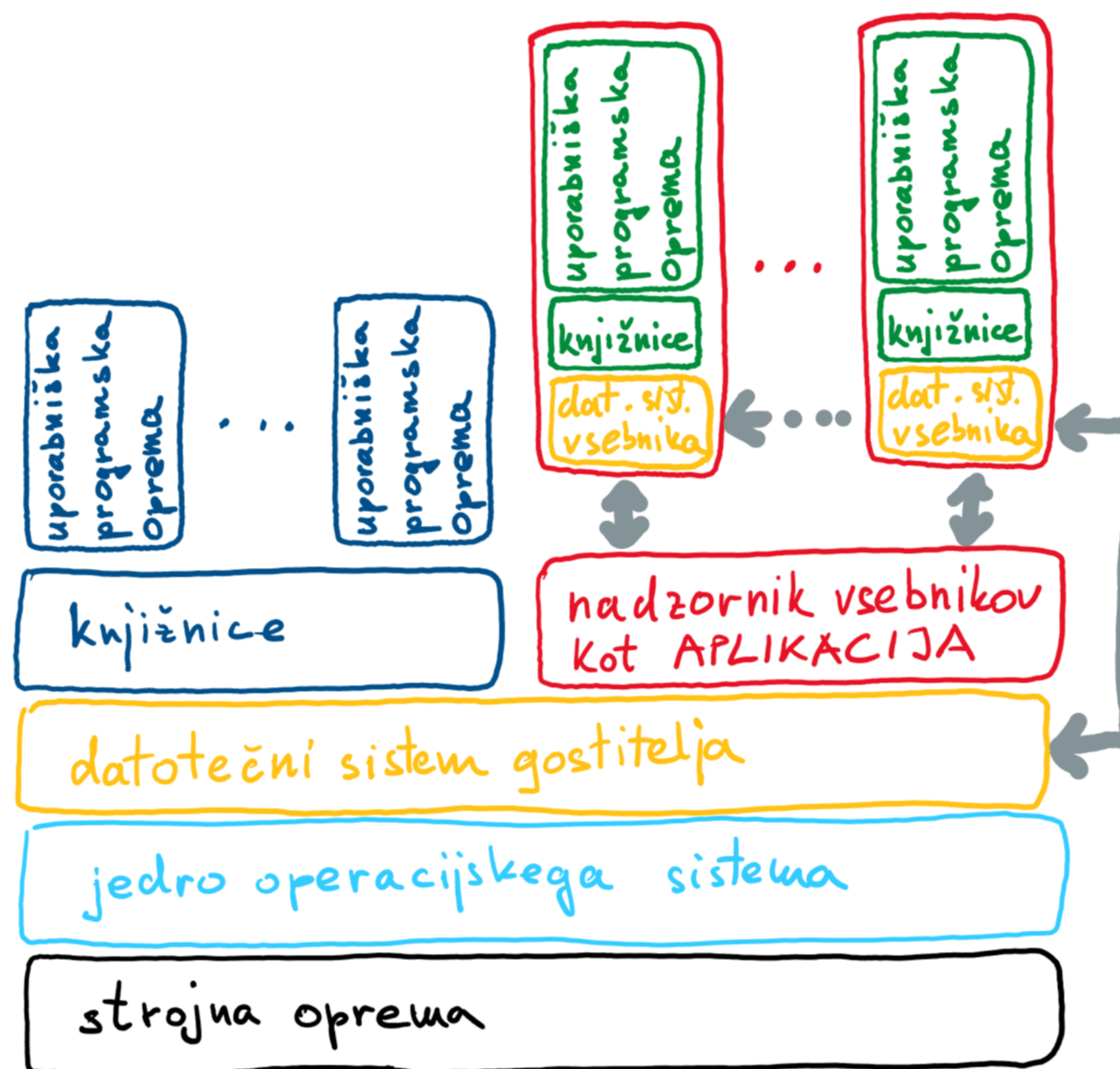
- Vsak **vsebnik** je shranjen **v eni sami datoteki**, ki vključuje knjižnice, orodja, uporabniško programsko opremo. Zato vsebnike enostavno prenašamo med računalniškimi sistemi.
- Uporabnik si **v vsebnike namesti poljubno programsko opremo** po svojih potrebah in željah. Pri delu v vsebniku ogrodje uporablja **enostaven varnostni model**, ki pravice uporabnika iz gostitelja prenese v vsebnik, v vsebniku pa ne dovoli spreminjanja pravic.
- Ogrodje Singularity daje **prednost integraciji pred izolacijo**, s čimer olajša uporabo posebne strojne opreme: grafičnih procesnih enot in drugih pospeševalnikov, nizkolatenčnih širokopasovnih omrežij in dostop do porazdeljenega datotečnega sistema.
- Vsebnike lahko **opremimo s kriptografskimi mehanizmi**, preko katerih uporabnik lahko preveri izvor vsebnika in se prepriča, da ni bil spremenjen.

Singularity

običajno ogrodje



ogrodje Singularity



Singularity

```
# GLAVA
Bootstrap: library
From: alpine:3

# RAZDELKI
%setup
    # ukaz, ki se bo izvedel na gostitelju po namestitvi OS v vsebniku
    (echo "====="; echo "modrec"; echo "=====") > info.txt

%files
    # datoteke, ki jih iz gostitelja prenesemo v vsebnik
    ./info.txt /info/

%environment
    # nastavitve spremenljivk okolja ob zagonu vsebnika
    export LC_ALL=C

%post
    # namestitveni koraki v vsebniku
    export LC_ALL=C

    apk update
    apk add fortune

%runscript
    # skripta, ki se izvede ob zagonu vsebnika
    fortune "$@"

%test
    # testna skripta
    cat /info/info.txt
    fortune
    test -x /usr/bin/fortune

%labels
    # poljubne oznake vsebnika kot pari oznaka – vrednost
    Author      uros (dot) lotric (at) fri (dot) uni (dash) lj (dot) si
    Container    modrec
    Version      1.0
    Description   Workshop advanced supercomputing (Superračunalištvo bolj zares)

%help
    # navodila za uporabo vsebnika
    Vsebnik vključuje program fortune za izpisovanje globokih misli
    Uporaba:
        singularity run modrec.sif
        ./modrec.sif
        singularity exec modrec.sif fortune
```